AMORAA

Third-Party API Implementation Report

Flutter frontend | Node.js/Express backend | MySQL

Client-ready implementation roadmap, account/document checklist, security plan, and testing reference

Prepared for	AMORAA project stakeholders and client account owners
Repository reviewed	D:\Projects\amora_ai | commit 2cb0ed59 | 14 August 2026
Document date	14 August 2026
Status	Final implementation planning reference - credentials not included
Classification	Client confidential

> **Important:** This report separates engineering work from client-owned prerequisites. Vendor pricing, SDK versions, product policies, and approval requirements must be revalidated at the start of each integration.

Contents

Document purpose and decision rules

This report converts the supplied 30-service inventory into an implementation plan grounded in the current AMORAA repository. It identifies what already exists, what must be completed or replaced, how each integration connects to Flutter, Express, and MySQL, and exactly what the client must provide before work can start.

> **Core decision rule:** 'All required integrations' does not mean installing every alternative. The project must choose one MMP (AppsFlyer or Adjust), one managed push approach (direct FCM or OneSignal), one managed chat path (Stream or Sendbird, or retain the existing chat), and one primary public-media stack (Cloudinary or AWS).

- P0 integrations are launch-critical only after the relevant channel/product is approved and the required client accounts are ready.
- P1 integrations are staged, roadmap-dependent, optional, or require a commercial decision before implementation.
- Backend/MySQL remains authoritative for user identity, payment verification, entitlement, KYC status, support mapping, and consent/audit records.
- No live secret values were copied into this report. The existing Backend/.env was checked only for presence/configuration state.

Executive assessment of the existing project

Flutter application	202 Dart source files; authenticated HTTP client with refresh-token rotation; secure token storage; feature repositories; Socket.IO chat; Razorpay UI; Google sign-in package.
Express backend	Express 4 + Sequelize + MySQL; JWT/refresh tokens; OTP/email/SMS adapters; Google token verification; FCM HTTP v1 adapter; Razorpay verification/webhook; media/KYC local storage; Socket.IO.
MySQL	23 ordered migrations in the latest internal audit; users, OTP/refresh tokens, profiles, discover, matches, chat, events, subscriptions/payments, notifications/devices/delivery, and identity verification.
Existing verification evidence	Internal audit dated 12 August 2026 reports Flutter analysis/build/tests, backend tests, applied migrations, restart persistence, and multi-user isolation passing for active internal flows; provider E2E remained credential-blocked.
Provider credentials	Firebase, Razorpay, Twilio, Google OAuth, and SMTP production credentials are absent/not configured in the checked environment. CORS is configured.

> **Release blockers before provider onboarding:** Android uses com.example.amora_ai and debug signing; iOS has no entitlements file; no google-services.json, GoogleService-Info.plist, or firebase_options.dart exists; the web shell has no tag/consent layer. Final identifiers, signing, store ownership, domains, privacy/legal documents, and secret management must be completed first.

Target integration architecture

Provider SDKs handle collection or user experience, while Express validates identity/business rules and MySQL stores the durable, auditable state. Webhooks are untrusted inputs until signature, timestamp, environment, ownership, and idempotency checks pass.

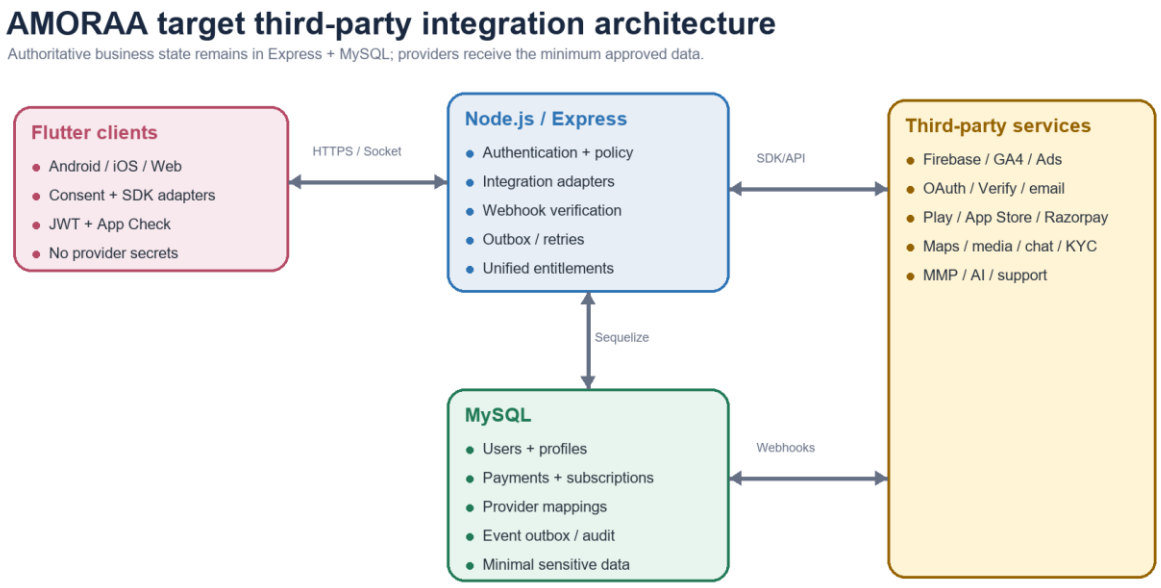

Figure 1. Target integration boundaries and systems of record.

Canonical cross-provider data contracts

- Event ID: globally unique UUID generated at the authoritative action; reused for browser/server deduplication.
- User identity: internal numeric user ID; external provider subjects live in ExternalIdentities; email/phone are never general analytics identifiers.
- Consent snapshot: category, version, jurisdiction, granted/withdrawn timestamps, and collection source available to every marketing delivery.
- Entitlement: MySQL Subscription/Payment state; store/provider callbacks trigger verification, not direct grants.
- Provider event ledger: provider event ID, signature result, payload hash, processing status, attempts, and immutable timestamps.
- Media: MediaAssets row points to provider asset ID/key, purpose/access class, checksum, moderation and deletion state.

> **Prohibited cross-provider data:** Aadhaar/KYC material, biometrics, OTPs, private message content, exact location, contact details, sexual orientation, religion, and sensitive dating preferences must not be sent to analytics, ads, MMPs, error tools, AI, or support by default.

Integration disposition matrix

#	Integration	P	Disposition	Current state
1	**Firebase Analytics**	P0	Implement	Not installed; no Firebase config files or FlutterFire initialization.
2	**Google Analytics 4 (Web)**	P0	Implement	No GA4 web stream, Google tag, consent layer, or measurement code in web/index.html.
3	**Google Tag Manager (Web)**	P0	Implement	No GTM container or dataLayer bootstrap in the web shell.
4	**Server-side Google Tag Manager**	P1	Implement after web GTM and consent design	Absent; Express sends no marketing events and no tagging domain exists.
5	**Meta Pixel + Conversions API**	P0	Implement for active Meta campaigns	Absent; no Meta Business assets, pixel, CAPI route, or consent mapping.
6	**Google Ads conversions**	P0	Implement for active Google Ads campaigns	Absent; no conversion actions, linked GA4 property, tag, enhanced conversions, or backend upload.
7	**TikTok Events API**	P1	Conditional - implement only when channel is funded	Absent; no pixel, event source, access token, or event mapping.
8	**AppsFlyer OR Adjust (Mobile Measurement Partner)**	P0	Choose exactly one MMP	Neither SDK exists; no ATT/advertising attribution flow, install table, or partner mappings.
9	**Branch**	P1	Decision gate after MMP selection	Absent; app has named routes but no universal/app links, AASA/DAL files, or link routing service.
10	**Firebase Cloud Messaging**	P0	Complete existing partial integration	Backend has FCM HTTP v1 adapter, UserDevices and NotificationDeliveries; Flutter has no firebase_messaging, token registration, APNs capability, or Firebase config.
11	**OneSignal (alternative)**	P1	Choose instead of direct FCM, not in addition	Absent; current backend is designed for direct FCM.
12	**Twilio Verify v2**	P0	Refactor existing Twilio Messaging OTP to Verify v2	twilio package and SMS adapter exist, but the backend generates/hashes OTPs locally and calls Programmable Messaging; no Verify Service SID.
13	**Google OAuth / Sign in with Google**	P0	Complete existing partial integration	Flutter google_sign_in and backend google-auth-library verification exist; GOOGLE_CLIENT_IDS is not configured and platform OAuth files/identifiers are incomplete.
14	**Sign in with Apple**	P0	Implement before iOS social-login release	Absent; no entitlement, Service ID, key, Apple callback, nonce, or ExternalIdentities table.
15	**Google Play Billing**	P0	Implement; replace mobile Razorpay for digital subscriptions	Absent. Current payment UI uses razorpay_flutter and MySQL subscriptions/payments; no Play products, purchase token fields, Developer API, or RTDN.

#	Integration	P	Disposition	Current state
16	**Apple StoreKit / App Store Server API**	P0	Implement	Absent; no IAP capability/products, transaction IDs, App Store keys, or server notification endpoint.
17	**Razorpay (Web payments/subscriptions)**	P1	Harden and reposition existing integration	Flutter mobile checkout, backend order/signature/payment fetch, webhook HMAC, Payments/PaymentEvents/Subscriptions already exist; credentials absent.
18	**Google Maps Platform**	P0	Implement	Android location permissions exist; DateSpotsMapScreen explicitly says map unavailable and uses static venue data. No Maps SDK/package/API keys/place schema.
19	**Cloudinary**	P0	Implement for public/non-regulated media	Profile photos are written under public /uploads; chat/KYC media use local private disk. No durable object storage/CDN/media asset model.
20	**AWS S3 + CloudFront (alternative)**	P1	Alternative/complement for regulated private media	Absent; local disk is not horizontally durable and no cloud IAM/bucket/CDN exists.
21	**Stream Chat OR Sendbird**	P0	Architectural decision - replace existing chat or retain it	AMORAA already has authenticated Socket.IO chat, MySQL conversations/messages/media/read state/presence/mute and tests. No managed chat SDK.
22	**Identity / KYC provider**	P0	Provider/legal decision required before production KYC	AMORAA accepts Aadhaar+selfie into private local storage and records pending status; no provider, liveness/OCR, admin review, or external decision callback.
23	**Sentry (optional)**	P1	Optional; recommended primarily for Express backend	Absent. Flutter has no Crashlytics/Sentry; Express logs to console only and has no release-aware APM.
24	**Firebase Crashlytics**	P0	Implement	Absent; main() does not initialize Firebase or install fatal/asynchronous error handlers.
25	**Firebase App Check**	P0	Implement after final signing/app IDs	Absent. Express accepts JWT-authenticated traffic without app attestation; Firebase resources/config are absent.
26	**Firebase Remote Config**	P1	Implement after analytics/governance	Absent; features and thresholds are compiled into Flutter/backend code.
27	**OpenAI API / approved LLM provider**	P1	Decision gate; backend-only implementation	No provider SDK/API. Active matching explanation is deterministic; AI coach/icebreaker UI exists but must not claim external AI results without integration.
28	**Transactional email provider (SES / SendGrid / approved)**	P0	Choose provider and complete existing SMTP abstraction	Nodemailer/SMTP adapter exists; production refuses to start without SMTP, but credentials are empty and only password-reset email is implemented.
29	**Transactional SMS / WhatsApp provider**	P1	Conditional; separate from Twilio Verify OTP	Generic sendSms sends OTP through Twilio Programmable Messaging only; no messaging service, templates, WhatsApp sender, consent

#	Integration	P	Disposition	Current state
				ledger, delivery callbacks, or category routing.
30	**Customer support / helpdesk provider**	P1	Choose provider and support operating model	Flutter has local FAQ/support UI; no ticket API, external help center, support accounts, SLA workflow, or SupportTickets table.

Client-owned prerequisite package

The fastest route is to assemble one controlled onboarding package before implementation begins. Accounts should be owned by the client, with development access granted by invitation; credentials should not be created under personal developer ownership.

Recommended account aliases	accounts@, engineering@, billing@, privacy@, security@, support@, marketing@ on the client domain; individual named admins with MFA rather than shared passwords.
Corporate identity	Legal entity name, registration/CIN, GST/PAN/tax details as applicable, registered/operating address, website, authorized signatory, directors/beneficial ownership if requested, bank/settlement proof, billing card/contact.
App ownership	Google Play Console organization; Apple Developer organization/App Store Connect; final package/bundle IDs; release signing/certificates; store listing IDs; test tracks/TestFlight.
Web/domain	Production and staging domains, DNS access, TLS/hosting, support/privacy/terms/refund URLs, subdomains for API, links, media, analytics, email return path.
Legal/privacy	Privacy Policy, Terms, Cookie Policy/CMP decision, Community Guidelines, refund/cancellation/subscription disclosures, consent text/versioning, DPDP/Aadhaar/KYC legal approval, DPO/grievance contact, retention/deletion schedule.
Operations	Marketing/ad owners, support agents/SLA, trust & safety/KYC reviewers, on-call/security contacts, finance/reconciliation owner, monthly budget/alerts, production approval matrix.

Account and document checklist by provider family

Google/Firebase/Maps/Ads/GTM	Corporate Google accounts, organization/billing, verified domains, final app IDs/signing fingerprints, OAuth consent assets, Ads advertiser/billing, GA/GTM publishers, Maps budgets.
Apple	Organization membership, D-U-N-S/legal enrollment if applicable, Account Holder, certificates/keys, App Store agreements, banking/tax, products/pricing, APNs and Server API keys.
Meta/TikTok	Verified business portfolios, ad accounts, payment methods, domains, pixels/datasets, campaign owners, data-sharing/consent approval.
Twilio/email/WhatsApp	Business verification, funding, number/sender ownership, India DLT/WABA/templates where applicable, sending domain DNS, from/reply-to, opt-in/out copy.
Payments	Razorpay KYC/legal/tax/bank/website/policies; Play/Apple merchant agreements

	and digital product catalog.
KYC	Vendor contract/KYB, legal/DPIA, exact Aadhaar/document method approval, retention/deletion, authorized signatory, webhook/live approval.
MMP/deep links	Vendor contract/billing, store IDs, ad-network access, ATT/privacy strategy, link domain/DNS, campaign taxonomy.
Cloud/media/chat/support/AI/observability	Vendor owner/billing/region/DPA, data residency/retention, role matrix, volumes/SLA, API projects/keys through secret manager, incident and deletion owners.

Pre-implementation technical gates

1. Replace Android applicationId com.example.amora_ai with the final reverse-domain ID and configure production signing; record SHA-1/SHA-256 fingerprints.
2. Finalize iOS bundle ID/App ID, provisioning, entitlements, APNs, associated domains, and StoreKit/Sign in with Apple capabilities.
3. Create dev/staging/prod domains and HTTPS API/webhook endpoints; lock CORS to explicit origins.
4. Adopt a managed secret store and environment promotion process; remove provider secrets from local .env distribution and CI logs.
5. Approve the event taxonomy, consent model, data retention/deletion, and provider data classification before analytics/ads/MMP SDK start.
6. Create generic integration outbox/webhook-event/mapping migrations and a worker/retry/dead-letter pattern before adding many adapters.
7. Define mobile-store versus web-payment product rules and one cross-platform entitlement state machine.
8. Freeze choose-one vendor decisions and avoid duplicate SDKs/destinations until the prior option is deliberately replaced.

Indicative implementation roadmap

Workstream	Indicative window	Exit outcome
Phase 0 - ownership and governance	Weeks 0-2+	Identifiers/signing/domains, secret management, accounts, legal/privacy/consent, vendor decisions, event and entitlement contracts. External approvals can extend this phase.
Phase 1 - foundation/observability	Weeks 2-4	Firebase Core/Analytics/Crashlytics/App Check monitor mode/Remote Config foundation; GA4/GTM consent layer; Sentry backend; integration outbox and webhook ledger.
Phase 2 - identity and communications	Weeks 3-6	Google/Apple sign-in, Twilio Verify, email, FCM token/delivery completion; transactional messaging decision.

Workstream	Indicative window	Exit outcome
Phase 3 - monetization	Weeks 5-9	Unified entitlement migration; Play Billing/RTDN; StoreKit/Server API/Notifications V2; Razorpay web hardening; finance reconciliation.
Phase 4 - product infrastructure	Weeks 7-12	Maps/Places, Cloudinary or AWS media, KYC provider, chat build-vs-buy decision/migration if selected.
Phase 5 - acquisition stack	Weeks 9-13	MMP, deep links, Meta/Google Ads, TikTok if active, server-side GTM if approved; test campaigns and reconciliation.
Phase 6 - roadmap services	Weeks 11-16+	AI pilot, support/helpdesk, advanced campaigns/experiments, hardening, penetration/load/DR, runbooks, production cutovers.

> **Schedule note:** The windows assume parallel Flutter/backend/infrastructure work after client prerequisites are ready. Store, ad, KYC, WhatsApp/DLT, payment, and vendor procurement approvals are external lead times and are not guaranteed by engineering.

Cross-provider testing and release gates

- Contract/unit tests for every adapter, signature verifier, event mapper, consent gate, and error translation.
- Sandbox/test-environment integration tests with provider-owned test tools and idempotent webhook fixtures.
- Physical Android/iOS device matrix for push, OAuth, deep links, ATT, location, purchases, media, KYC, and account switching.
- Reconciliation from provider dashboard/API to MySQL and outbox; document expected reporting delay/discrepancy.
- Security tests: secret scanning, wrong-audience/wrong-environment tokens, signature/replay/out-of-order webhooks, object access, rate abuse, and least privilege.
- Privacy tests: consent denied/withdrawn, data deletion/export, log scrubbing, SDK identity reset, retention expiry, and prohibited-field scans.
- Operational tests: provider outage, timeouts/retry/dead-letter, quota/cost alarms, credential rotation, webhook backlog, rollback/kill switch, and incident notifications.
- Production smoke tests use client-approved test accounts/amounts/content and require business, finance, legal/privacy, and engineering sign-off where relevant.

Detailed integration implementation plans

The following sections are the implementation reference for each item in the supplied master inventory. Each section includes current-state evidence, cross-layer changes, configuration, webhooks, ownership, sequence, testing, and security controls.

1. Firebase Analytics

Canonical first-party mobile/app analytics, funnels, cohorts, retention, and user properties.

Priority	P0
Disposition	Implement
Current AMORAA state	Not installed; no Firebase config files or FlutterFire initialization.
Client dependency	High - final app IDs, Firebase/GCP ownership, privacy decisions; 1-3 client days.
Engineering size	M
Primary layer	Flutter (Android/iOS/Web) + reporting

> **Recommendation:** Use Firebase Analytics as the canonical app event collector. The same event dictionary must feed GA4 and the chosen MMP; do not create competing names for the same signup, match, subscription, or retention events.

Implementation and connection to AMORAA

SDK/API: FlutterFire firebase_core and firebase_analytics; Firebase/GA4 app data streams.

Flutter/web changes: Initialize Firebase before AuthService, add consent-aware analytics service, screen-view observer, stable user ID after login, and typed event wrappers. Exclude chat text, exact location, Aadhaar/KYC media, and other sensitive attributes.

Node.js/Express changes: No Firebase SDK is required for normal client events. Backend will emit authoritative server events (payment, entitlement, verification) through a shared event outbox to GA4 Measurement Protocol/server routing only after consent rules are satisfied.

MySQL requirements: Add AnalyticsEventOutbox (eventId, userId nullable, eventName, payload JSON, consentState, destinationState, occurredAt) or reuse a generic IntegrationOutbox. Store no advertising identifiers in Users.

Configuration/environment: FIREBASE_* platform configuration via generated firebase_options.dart; release identifiers are non-secret. Server measurement secrets live only in the secret manager, never Flutter.

Webhooks/callbacks: None for the client SDK. Server event delivery uses provider HTTP responses and retry/dead-letter tracking.

Development team will implement

- Define a versioned AMORAA event and user-property taxonomy with owners and PII classification.
- Instrument acquisition, signup, verification, onboarding, discover, match, chat-open, event registration, paywall, purchase, cancel, and retention milestones.

- Add debug-mode verification and a data-quality dashboard; document which events are client-observed versus server-authoritative.
- Add consent/opt-out behavior and account deletion/reset handling.

Client must provide or complete

- Provide a company-controlled Google account and create/invite the team to a Firebase project and linked GA4 property.
- Approve the event taxonomy, conversion definitions, retention period, data-sharing settings, and whether advertising features are enabled.
- Provide the final Android application ID, iOS bundle ID, App Store ID when available, and production web domain.
- Provide approved Privacy Policy/Cookie Policy wording and analytics consent requirements by launch market.

Implementation sequence

9. Finalize identifiers and create dev/staging/prod Firebase apps.
10. Install FlutterFire, generate per-platform config, and initialize consent defaults.
11. Implement typed analytics facade and event schema validation.
12. Instrument critical flows and server-authoritative event forwarding.
13. Validate DebugView, user isolation, opt-out, and dashboard funnels before release.

Testing and acceptance

- Firebase DebugView confirms exact names/parameters once per action on Android, iOS, and web.
- Automated unit tests verify event mapping and block prohibited PII fields.
- Logout/account-switch tests clear user identity; consent-denied tests send no disallowed analytics.
- Reconcile signup/purchase counts against MySQL within an agreed tolerance.

Security and privacy controls

- Use pseudonymous internal user IDs only after authentication; never send phone, email, Aadhaar, chat content, photos, or precise coordinates.
- Restrict console roles and enable MFA; separate production from non-production.
- Document deletion/retention behavior and consent evidence.

Official implementation reference: Firebase for Flutter setup

2. Google Analytics 4 (Web)

Web acquisition, audience, conversion, journey, and funnel reporting for the Flutter web property.

Priority	P0
Disposition	Implement
Current AMORAA state	No GA4 web stream, Google tag, consent layer, or measurement code in web/index.html.
Client dependency	High - GA ownership, domain, consent/legal decisions; 1-3 client days.
Engineering size	M
Primary layer	Flutter Web + GA4

> **Recommendation:** Create one GA4 property with distinct production web and app streams where governance permits. Treat Firebase Analytics as the app collector and GA4/GTM as the web collector, with a shared event dictionary and deduplication rules.

Implementation and connection to AMORAA

SDK/API: Google tag/GA4 through GTM for web; Firebase Analytics for Flutter app streams; Measurement Protocol only for authoritative server events.

Flutter/web changes: Add consent default before tag load, route/screen tracking for Flutter navigation, virtual page views, campaign parameters, and typed dataLayer events. Add canonical URL/referrer handling for SPA navigation.

Node.js/Express changes: Issue server events only when client identifiers/consent are valid. Generate a stable event_id for purchase/signup deduplication and keep secrets server-side.

MySQL requirements: Use AnalyticsEventOutbox and optional WebAttributionTouch (userId, clientIdHash, campaign fields, consent, first/last touch) with strict retention; do not persist raw cookies unless approved.

Configuration/environment: GA4_MEASUREMENT_ID is public web config; GA4_API_SECRET and measurement endpoint configuration are backend secrets. Configure allowed domains and referral exclusions in GA.

Webhooks/callbacks: No inbound webhook. Measurement Protocol delivery must record provider response/retry state.

Development team will implement

- Create web data stream and GTM-delivered GA configuration.
- Instrument virtual page views and conversion events in the Flutter router/dataLayer bridge.
- Configure cross-domain/referral rules, internal-traffic filters, retention, and BigQuery export decision.
- Build server-event reconciliation and duplicate suppression.

Client must provide or complete

- Provide GA4 property owner access or authorize creation under the corporate Google organization.
- Provide final production/staging domains and DNS/hosting owner.
- Approve key events/conversions, audiences, retention, data-sharing, and consent-mode choices.
- Provide cookie/privacy notices and the marketing/legal owner who can approve analytics usage.

Implementation sequence

14. Create property/data stream and naming standard.
15. Implement consent-first loader and Flutter web dataLayer bridge.
16. Configure conversions and referral/domain rules.
17. Verify in Realtime/DebugView and reconcile against backend records.

Testing and acceptance

- Tag Assistant and GA DebugView show SPA route changes without duplicates.
- UTM/gclid persistence survives login only within approved consent scope.
- Consent denied/granted/withdrawn scenarios behave as documented.
- Purchase and signup totals reconcile to MySQL/outbox records.

Security and privacy controls

- API secrets never appear in web assets.
- Default consent state follows the approved jurisdictional policy.
- No sensitive dating preferences, messages, verification details, or raw identifiers enter GA.

Official implementation reference: GA4 consent types

3. Google Tag Manager (Web)

Governed deployment of GA4, ad pixels, consent signals, and marketing tags without rebuilding the Flutter app.

Priority	P0
Disposition	Implement
Current AMORAA state	No GTM container or dataLayer bootstrap in the web shell.
Client dependency	High - GTM admin access and marketing governance; 1-2 client days.
Engineering size	M
Primary layer	Flutter Web shell + marketing operations

> **Recommendation:** Use GTM only for web tags. Keep business-event generation in typed Flutter/backend code and expose a controlled dataLayer contract; do not allow ad-hoc tags to read the entire page or application state.

Implementation and connection to AMORAA

SDK/API: Google Tag Manager web container and dataLayer API.

Flutter/web changes: Insert the approved GTM loader and consent defaults in web/index.html, then dispatch sanitized route and conversion objects from Flutter through a JS interop adapter.

Node.js/Express changes: No direct runtime dependency. Backend supplies authoritative event IDs and, where approved, first-party campaign context to the web client/server-side tag path.

MySQL requirements: No GTM-specific table. Reuse event outbox and consent/audit records.

Configuration/environment: GTM_CONTAINER_ID per environment; container environments/auth strings for staging preview; CSP allow-list updates.

Webhooks/callbacks: None. Tags may call approved endpoints only; all vendor endpoints are documented in the tag register.

Development team will implement

- Create a versioned dataLayer schema with a field allow-list and examples.
- Implement consent initialization before GTM and SPA route events after Flutter navigation.
- Configure workspaces, environments, approvals, naming, folders, and publishing workflow.
- Add Content Security Policy and automated container export backups.

Client must provide or complete

- Provide corporate GTM account/container owner access and nominate marketing publishers/approvers.

- Approve the tag register, consent categories, and who may publish production changes.
- Provide production/staging domains and access to modify the web host/CSP.
- Provide all vendor pixel IDs only after the corresponding business accounts are verified.

Implementation sequence

18. Create account/containers and least-privilege roles.
19. Define consent and dataLayer contracts.
20. Install/verify on staging.
21. Configure tags/triggers/variables and approvals.
22. Publish production container with rollback/export.

Testing and acceptance

- GTM Preview shows only approved fields and triggers.
- Tag Assistant confirms tags are suppressed until the relevant consent.
- SPA navigation and duplicate-fire regression tests pass.
- CSP and page-load performance remain within agreed thresholds.

Security and privacy controls

- No secrets in GTM variables; public pixel IDs only.
- Limit custom HTML tags and require review for templates/network destinations.
- Maintain container export, change log, and emergency rollback owner.

Official implementation reference: Google Tag Manager web help

4. Server-side Google Tag Manager

First-party server event routing, transformation, validation, and controlled forwarding to analytics/advertising destinations.

Priority	P1
Disposition	Implement after web GTM and consent design
Current AMORAA state	Absent; Express sends no marketing events and no tagging domain exists.
Client dependency	High - GCP/billing/DNS/consent ownership; typically 3-10 client days plus DNS propagation.
Engineering size	L
Primary layer	Backend/web + managed tagging infrastructure

> **Recommendation:** Deploy only after the client approves hosting cost and data governance. Use a dedicated first-party subdomain and retain the Express outbox as the system of record; sGTM is a router, not the entitlement/payment authority.

Implementation and connection to AMORAA

SDK/API: GTM server container on Cloud Run/App Engine or approved managed host; GA4 client and vendor server tags.

Flutter/web changes: Point approved GA4/GTM traffic to the first-party tagging domain and preserve consent/event_id parameters.

Node.js/Express changes: Add signed/allow-listed event ingress or direct server-container requests from the integration worker; validate schema, consent, timestamps, and retry behavior.

MySQL requirements: AnalyticsEventOutbox/IntegrationDeliveryAttempt plus provider event IDs and dead-letter status.

Configuration/environment: SGTM_BASE_URL, SGTM_AUTH_SECRET or network identity, GTM server container config, GCP project/region; DNS CNAME for analytics.<domain>.

Webhooks/callbacks: Outbound routing to GA4, Google Ads, Meta, TikTok as approved. No anonymous inbound business events.

Development team will implement

- Provision a production-grade multi-instance server container and custom domain.
- Implement transformation/redaction templates and destination allow-list.
- Connect backend outbox with idempotent delivery and monitoring.
- Document operating cost, scaling, logging, rollback, and ownership.

Client must provide or complete

- Approve server-side tagging scope, monthly hosting budget, GCP billing project, and data-processing destinations.
- Provide DNS access for the first-party tagging subdomain and TLS validation.
- Provide GTM/GA/Ads/Meta/TikTok administrator access and legal approval for server event sharing.
- Nominate a marketing-operations owner for ongoing container publishing.

Implementation sequence

23. Approve architecture and hosting model.
24. Provision server container and custom domain.
25. Implement redaction/client templates and Express outbox delivery.
26. Connect one destination at a time, validate, then expand.
27. Load/failover test and document runbook.

Testing and acceptance

- Preview/debug confirms field transformations and consent propagation.
- Duplicate, late, and malformed events are rejected or deduplicated.
- Outage/retry/dead-letter tests preserve events without double conversions.
- DNS, TLS, autoscaling, log redaction, and monitoring checks pass.

Security and privacy controls

- Authenticate backend-to-container traffic and restrict admin roles.
- Redact IP/PII and drop prohibited dating/KYC/chat fields before vendors.
- Set short log retention and alert on new destinations or template changes.

Official implementation reference: Server-side GTM overview

5. Meta Pixel + Conversions API

Meta campaign measurement, optimization, retargeting, and deduplicated browser/server conversion signals.

Priority	P0
Disposition	Implement for active Meta campaigns
Current AMORAA state	Absent; no Meta Business assets, pixel, CAPI route, or consent mapping.
Client dependency	Very High - verified Meta business, dataset ownership, legal consent, ad account; often 1-4 weeks.
Engineering size	L
Primary layer	Web + backend

> **Recommendation:** Use Pixel for consented browser events and CAPI for authoritative signup/subscription events with the same event_name and event_id. Do not place Meta SDK/pixel in the mobile app unless the MMP and privacy plan explicitly require it.

Implementation and connection to AMORAA

SDK/API: Meta Pixel through GTM; Meta Conversions API/Marketing API from backend or sGTM.

Flutter/web changes: Generate consented PageView/ViewContent/Lead events from the web dataLayer and capture only approved click/cookie parameters.

Node.js/Express changes: Create CAPI adapter for CompleteRegistration, Subscribe, Purchase, Cancel/eligible lifecycle events; hash normalized contact fields only when legally approved and needed for match quality.

MySQL requirements: MarketingEventOutbox (or shared outbox), eventId, sourceUrl, consent snapshot, pixel/dataset destination status; store access tokens only in secrets manager.

Configuration/environment: META_PIXEL_ID, META_DATASET_ID, META_CAPI_ACCESS_TOKEN, META_TEST_EVENT_CODE (non-prod), META_API_VERSION.

Webhooks/callbacks: Outbound CAPI. Optional Meta lead/webhook ingestion is out of scope unless the client later requests it.

Development team will implement

- Map AMORAA events to Meta standard/custom events and define deduplication.
- Implement Pixel via GTM and backend CAPI adapter with test-event mode.
- Configure domain verification, Aggregated Event Measurement priorities where applicable, and diagnostics.
- Add consent gating, retry, match-quality monitoring, and deletion workflow.

Client must provide or complete

- Create/verify Meta Business Portfolio, ad account, dataset/pixel, and corporate domain; invite development and marketing roles.
- Provide payment method/ad billing and nominate business/admin owners.
- Approve the exact event list, audience/retargeting policy, contact-data hashing, retention, and consent language.
- Provide Privacy Policy/Cookie Policy and domain DNS access for verification.

Implementation sequence

28. Verify business/domain and create dataset.
29. Approve event/data-sharing matrix.
30. Install Pixel in staging and implement CAPI test events.
31. Validate deduplication/diagnostics and consent behavior.
32. Enable production campaigns after marketing sign-off.

Testing and acceptance

- Meta Test Events receives browser and server events with one conversion after deduplication.
- Event Match Quality is reviewed without adding prohibited data.
- Consent denied/withdrawn blocks future ad signals.
- Refund/cancel and duplicate webhook cases do not inflate purchase conversions.

Security and privacy controls

- Keep access token server-side and rotate through the client-owned business account.
- Hash only normalized approved fields; hashing does not remove the need for consent/legal basis.
- Never transmit sexual orientation, religion, dating preferences, KYC, messages, or precise location.

Official implementation reference: Meta Conversions API

6. Google Ads conversions

Google Ads conversion measurement, campaign optimization, and audience activation for approved acquisition programs.

Priority	P0
Disposition	Implement for active Google Ads campaigns
Current AMORAA state	Absent; no conversion actions, linked GA4 property, tag, enhanced conversions, or backend upload.
Client dependency	Very High - Ads account/billing, GA/GTM access, consent/legal decisions; 2-10 client days.
Engineering size	M/L
Primary layer	Web + backend

> **Recommendation:** Prefer GA4 key-event import for broad web measurement and direct Google Ads conversion/server tags for revenue-critical events where deduplication and enhanced conversions are approved.

Implementation and connection to AMORAA

SDK/API: Google Ads conversion tags through GTM/sGTM; GA4 linking/import; optional Google Ads API for offline/enhanced conversion uploads.

Flutter/web changes: Send consented gclid/gbraid/wbraid and conversion event IDs through the web dataLayer without exposing backend secrets.

Node.js/Express changes: Persist approved click identifiers with attribution expiry, upload authoritative conversions/refunds, and record Google response/job status.

MySQL requirements: AttributionTouches and MarketingEventOutbox with click ID, event ID, conversion time/value/currency, consent state, destination status, expiry.

Configuration/environment: GOOGLE_ADS_CUSTOMER_ID, CONVERSION_ID/LABEL or action resource name, developer/OAuth credentials if API uploads are used; GA/GTM identifiers.

Webhooks/callbacks: Outbound tag/API uploads. Google Ads does not drive AMORAA entitlement state.

Development team will implement

- Create conversion mapping and value rules for lead, verified signup, subscription, and renewal/cancel events.
- Link GA4/Ads and implement tags or API upload with one authoritative source per conversion.
- Implement click-ID capture/retention and consent-mode signals.
- Build reconciliation dashboard and campaign launch checklist.

Client must provide or complete

- Provide Google Ads admin access, advertiser verification details, billing profile/payment method, and campaign owner.
- Provide GA4/GTM admin access and approve linked-product data sharing.
- Approve conversion values, attribution windows, customer-data usage, audiences, and consent requirements.
- Provide domain ownership/DNS access and legal/privacy documents.

Implementation sequence

33. Complete advertiser/account verification and link products.
34. Approve conversion source/value model.
35. Implement web and server signals with deduplication.
36. Validate in Tag Assistant/Ads diagnostics.
37. Launch campaigns only after reconciliation sign-off.

Testing and acceptance

- Test conversions appear with correct action, value, currency, timestamp, and transaction ID.
- Duplicate client/server events count once.
- Consent-mode and click-ID expiry tests pass.
- Refund/cancel corrections and backend reconciliation are verified.

Security and privacy controls

- OAuth/API credentials remain server-side; use least-privilege client-owned accounts.
- Obtain required consent before enhanced-conversion/customer-data use.
- Prohibit sensitive profile/KYC/chat data in ads or audiences.

Official implementation reference: Server-side Google Ads conversions

7. TikTok Events API

TikTok campaign measurement and optimization when TikTok is an approved acquisition channel.

Priority	P1
Disposition	Conditional - implement only when channel is funded
Current AMORAA state	Absent; no pixel, event source, access token, or event mapping.
Client dependency	Very High - TikTok Business/Ads account, billing, legal approval; 1-4 weeks.
Engineering size	M/L
Primary layer	Web + backend

> **Recommendation:** Defer until the client confirms an active TikTok campaign plan. When used, implement Pixel plus Events API with shared event_id deduplication and the same consent/data-minimization controls as Meta.

Implementation and connection to AMORAA

SDK/API: TikTok Pixel through GTM and TikTok Events API 2.0 direct/sGTM integration.

Flutter/web changes: Emit approved web events and click/cookie context only after consent.

Node.js/Express changes: Send authoritative registration/subscription events with event_id, event_time, context, and approved match keys; record retries/diagnostics.

MySQL requirements: Reuse MarketingEventOutbox and AttributionTouches; no TikTok-specific user fields.

Configuration/environment: TIKTOK_PIXEL_CODE, TIKTOK_EVENTS_ACCESS_TOKEN, TIKTOK_API_BASE/VERSION, TIKTOK_TEST_EVENT_CODE.

Webhooks/callbacks: Outbound Events API only for this scope.

Development team will implement

- Define the TikTok event/match-key matrix with marketing/legal.
- Configure Pixel and direct/server events with deduplication.
- Implement retry, diagnostics, consent enforcement, and destination kill switch.
- Document campaign QA and ongoing signal-quality ownership.

Client must provide or complete

- Provide verified TikTok Business Center, Ads Manager account, pixel/event source, billing, and admin invitations.
- Approve that TikTok is an active channel and define campaign/conversion objectives.
- Approve match keys, audience policy, consent text, and privacy/data-sharing terms.

- Provide domain verification access if requested by the platform.

Implementation sequence

38. Complete business/ad setup.
39. Approve data-sharing matrix.
40. Implement Pixel and Events API in test mode.
41. Validate deduplication/diagnostics.
42. Enable production destination for campaign launch.

Testing and acceptance

- TikTok Events Manager diagnostics show browser/server events and deduplication.
- Consent-denied and data-field allow-list tests pass.
- Retries/duplicates/late events do not inflate conversions.
- Counts reconcile against MySQL and MMP reporting.

Security and privacy controls

- Access token is a backend secret; disable destination when no campaigns are active.
- No sensitive dating, KYC, chat, or precise location data.
- Legal review is required before match keys or custom audiences.

Official implementation reference: TikTok Events API overview

8. AppsFlyer OR Adjust (Mobile Measurement Partner)

Install attribution, campaign source, deep-link attribution, in-app event measurement, and mobile marketing postbacks.

Priority	P0
Disposition	Choose exactly one MMP
Current AMORAA state	Neither SDK exists; no ATT/advertising attribution flow, install table, or partner mappings.
Client dependency	Very High - vendor contract, store listings/IDs, ad accounts, privacy decisions; 1-4 weeks.
Engineering size	L
Primary layer	Flutter Android/iOS + backend

> **Recommendation:** Run a commercial/technical selection and implement one MMP only. Prefer the selected MMP's native deep-link product first; add Branch only if it supplies a documented capability the MMP cannot meet.

Implementation and connection to AMORAA

SDK/API: appsflyer_sdk plus OneLink, or adjust_sdk plus TrueLink/deep links; vendor S2S API/postbacks.

Flutter/web changes: Initialize after consent policy, set internal customer ID, record standardized lifecycle/revenue events, handle attribution/deferred links, ATT/SKAdNetwork flow, and logout identity reset.

Node.js/Express changes: Receive verified postbacks where needed, send authoritative revenue/subscription events, reconcile store/Razorpay transactions, and handle deletion/forget requests.

MySQL requirements: AttributionInstalls, AttributionTouches, ProviderPostbackEvents, and event outbox with provider IDs, campaign dimensions, consent, first/last touch, and expiry.

Configuration/environment: MMP_DEV_KEY/APP_TOKEN, iOS App ID, Android package, environment, deep-link domain/template IDs, S2S auth token; public app token may be in app only per vendor guidance.

Webhooks/callbacks: Attribution/install postbacks, partner postbacks, and data-deletion callbacks as contracted; all signed/authenticated and idempotent.

Development team will implement

- Lead an AppsFlyer-vs-Adjust decision against pricing, channel coverage, deep linking, fraud, privacy, raw data, and support.
- Create the unified mobile event map and revenue-source rules.
- Implement SDK, consent/ATT, deep links, S2S events, and postback ingestion.
- Build attribution QA, discrepancy dashboard, deletion, and account-switch controls.

Client must provide or complete

- Select and purchase one MMP plan; provide organization owner/admin invitations and billing contact.
- Provide final Android package, iOS bundle/App Store ID, store access, ad-network accounts, and campaign naming conventions.
- Approve ATT prompt strategy, consent wording, data retention/export, raw-data access, partner postbacks, and fraud settings.
- Provide the marketing attribution owner and acceptable discrepancy thresholds.

Implementation sequence

43. Vendor selection and contract/DPA.
44. Create apps, event map, partner connections, and deep-link domain.
45. Implement sandbox/dev SDK and consent/ATT behavior.
46. Connect S2S revenue/postbacks and reconcile.
47. Run live test campaigns before production sign-off.

Testing and acceptance

- Fresh install, reinstall, organic, paid, deferred deep link, and account-switch cases on physical devices.
- Sandbox/live-test campaign attribution appears under correct source/campaign.
- Revenue equals verified server transactions; SDK/S2S duplicates are suppressed.
- Opt-out/deletion and ATT-denied flows behave per policy.

Security and privacy controls

- Keep S2S tokens and reporting credentials server-side.
- Treat advertising IDs and attribution data as personal data; apply retention and access controls.
- Do not send sensitive profile traits, KYC, messages, or photos as event parameters.

Official implementation reference: AppsFlyer SDK getting started

9. Branch

Branded direct/deferred deep links, campaign links, sharing links, and cross-platform routing.

Priority	P1
Disposition	Decision gate after MMP selection
Current AMORAA state	Absent; app has named routes but no universal/app links, AASA/DAL files, or link routing service.
Client dependency	High - link domain/DNS, store IDs, Branch plan; 3-10 client days.
Engineering size	L
Primary layer	Flutter + web/domain + backend

> **Recommendation:** Do not add Branch automatically if AppsFlyer OneLink or Adjust deep links meet the requirement. If selected, define a narrow route allow-list for profiles/events/referrals and never encode personal data in URLs.

Implementation and connection to AMORAA

SDK/API: flutter_branch_sdk, Branch dashboard/link API, Android App Links, iOS Universal Links, web fallback.

Flutter/web changes: Initialize link listener before routing, map canonical link payloads to safe named routes, handle cold/warm/deferred opens, logout identity reset, and invalid-link fallback.

Node.js/Express changes: Optionally create signed/authorized share links and store referral/link ownership; validate any reward claim independently of link parameters.

MySQL requirements: DeepLinks/ReferralLinks (linkId, ownerUserId, targetType/id, campaign, status, expiresAt) and AttributionTouches only if programmatic links/referrals are used.

Configuration/environment: BRANCH_KEY per environment, BRANCH_SECRET server-only, BRANCH_LINK_DOMAIN, iOS team/bundle/App Store IDs, Android package/signing fingerprints.

Webhooks/callbacks: Optional Branch webhook/export callbacks; signed/authenticated and deduplicated.

Development team will implement

- Decide overlap with MMP deep-link capability and document the reason for Branch.
- Configure link domains, AASA/assetlinks, route allow-list, and fallbacks.
- Implement Flutter cold/warm/deferred routing and backend link creation if needed.
- Add link abuse controls and campaign/share QA.

Client must provide or complete

- Approve Branch as an additional vendor and plan; provide account owner/admin access.

- Provide final bundle/package/store IDs, Android release signing SHA-256, Apple Team ID, and DNS access.
- Approve branded link domain, link lifespan, referral rules, and fallback pages.
- Provide Privacy Policy/DPA approval and campaign naming owner.

Implementation sequence

48. Confirm vendor necessity.
49. Configure apps/domains/store destinations.
50. Implement safe link schema/routing.
51. Implement API-generated links/referrals if required.
52. Test every install/open/fallback matrix.

Testing and acceptance

- Installed, not-installed, cold-start, warm-start, logged-out, expired, and invalid links.
- Android App Links and iOS Universal Links validate against production signing/team IDs.
- No route permits unauthorized access to private profiles/chats.
- Referral/link reward cannot be replayed or self-awarded.

Security and privacy controls

- Never put phone, email, Aadhaar, tokens, or sensitive profile fields in URLs.
- Backend re-authorizes target access after navigation.
- Use separate test/live keys and clear Branch identity on logout.

Official implementation reference: Branch Flutter SDK reference

10. Firebase Cloud Messaging

Push notifications for transactional events, safety, re-engagement, messages, matches, events, and subscription status.

Priority	P0
Disposition	Complete existing partial integration
Current AMORAA state	Backend has FCM HTTP v1 adapter, UserDevices and NotificationDeliveries; Flutter has no firebase_messaging, token registration, APNs capability, or Firebase config.
Client dependency	High - Firebase project, APNs key, app IDs/signing; 2-10 client days.
Engineering size	M/L
Primary layer	Flutter + backend

> **Recommendation:** Complete FCM because the backend delivery ledger already exists. Add OneSignal only if the client explicitly buys its campaign UI/segmentation and accepts replacing the current provider adapter.

Implementation and connection to AMORAA

SDK/API: firebase_messaging + firebase_core; Firebase Admin SDK/HTTP v1; APNs authentication key uploaded to Firebase.

Flutter/web changes: Request permission contextually, obtain/rotate token, POST /api/devices, unregister on logout, handle foreground/background/terminated messages, route deep links, and show local foreground notifications where required.

Node.js/Express changes: Replace raw credential handling with a shared Firebase Admin initialization where practical, support topic-free per-user sends, scheduled retry worker, invalid-token cleanup, quiet hours, collapse keys, and environment separation.

MySQL requirements: Existing UserDevices, Notifications, NotificationPreferences, NotificationDeliveries are suitable; add provider, appEnvironment, locale/timezone, token fingerprint, nextAttemptAt if needed.

Configuration/environment: FIREBASE_PROJECT_ID plus service-account/workload identity; GOOGLE_APPLICATION_CREDENTIALS or secret-manager fields; APNs key configured in Firebase; no service-account secret in Flutter.

Webhooks/callbacks: FCM delivery is outbound; token refresh is a client callback. Delivery metrics export is optional and privacy-reviewed.

Development team will implement

- Add FlutterFire/FCM configuration and lifecycle handlers.
- Register/rotate/unregister tokens using the existing device endpoints.
- Harden backend retries, invalidation, quiet hours, deep-link payload schema, and observability.

- Create notification copy/category matrix and operational runbook.

Client must provide or complete

- Provide owner access to a client-controlled Firebase project and approve dev/staging/prod app registrations.
- Provide final app identifiers and Apple Developer access/APNs .p8 key, Key ID, and Team ID; never email the private key in plaintext.
- Approve notification categories, opt-in copy, quiet hours, retention, and re-engagement policy.
- Provide physical Android/iOS test devices and production signing/store access.

Implementation sequence

53. Finalize app IDs and Firebase apps.
54. Configure Android/iOS/APNs and Flutter SDK.
55. Connect token lifecycle to /api/devices.
56. Harden backend delivery/retry and payload routes.
57. Run device matrix and production test push.

Testing and acceptance

- Foreground/background/terminated delivery on physical Android/iOS devices.
- Permission denied/later-granted, token rotation, reinstall, logout, and account-switch cases.
- Invalid tokens deactivate devices; retries do not duplicate user-visible alerts.
- Quiet hours/category preferences/mute and deep-link authorization work.

Security and privacy controls

- Service-account/APNs secrets stay in managed secrets and client-owned consoles.
- Notification bodies reveal no sensitive dating/KYC content on lock screens unless explicitly approved.
- Backend verifies user/device ownership and rate limits registration.

Official implementation reference: FCM for Flutter

11. OneSignal (alternative)

Managed push/in-app messaging, segmentation, campaigns, and marketer-facing engagement workflows.

Priority	P1
Disposition	Choose instead of direct FCM, not in addition
Current AMORAA state	Absent; current backend is designed for direct FCM.
Client dependency	Very High - vendor selection/contract plus Firebase/APNs accounts; 1-3 weeks.
Engineering size	L
Primary layer	Flutter + backend + OneSignal

> **Recommendation:** Stay with direct FCM unless the client needs non-developer campaign creation, journeys, in-app messages, or advanced segmentation enough to justify migration and subscription cost.

Implementation and connection to AMORAA

SDK/API: onesignal_flutter, OneSignal REST API, FCM/APNs credentials.

Flutter/web changes: Initialize OneSignal, request permission, set AMORAA user ID as External ID after login, clear identity at logout, handle click/deep links, tags, and consent.

Node.js/Express changes: Create OneSignal provider adapter and API client; prevent duplicate delivery from current FCM adapter; map NotificationDeliveries to OneSignal message IDs.

MySQL requirements: Reuse UserDevices/NotificationDeliveries; add provider subscription ID/external ID and campaign metadata if selected.

Configuration/environment: ONESIGNAL_APP_ID public; ONESIGNAL_REST_API_KEY server-only; FCM service account and APNs .p8 configured in OneSignal.

Webhooks/callbacks: Delivery/click events and identity subscription callbacks as selected; verify authenticity and deduplicate.

Development team will implement

- Prepare FCM-vs-OneSignal decision memo and migration plan.
- Implement SDK identity, token/subscription lifecycle, deep links, and backend adapter.
- Configure segments/templates/journeys with governance and no duplicate FCM sends.
- Map delivery outcomes to the existing ledger and preferences.

Client must provide or complete

- Approve OneSignal selection, commercial plan, DPA, organization owner, and billing.

- Provide OneSignal admin access plus Firebase/APNs credentials through secure transfer.
- Approve who can send campaigns, audience rules, copy templates, frequency caps, and in-app messages.
- Nominate marketing operations and support owners.

Implementation sequence

58. Approve vendor and disable direct-send migration conflict.
59. Configure apps/platform credentials.
60. Integrate Flutter identity/consent and backend adapter.
61. Migrate templates/segments and test.
62. Cut over with duplicate-delivery monitoring.

Testing and acceptance

- OneSignal test subscriptions for Android/iOS and authenticated user mapping.
- Logout/account switch cannot receive another user's messages.
- Direct FCM path is disabled during OneSignal delivery.
- Click/deep-link, opt-out, frequency cap, and delivery callback tests pass.

Security and privacy controls

- REST API key never ships in Flutter.
- External IDs are non-secret internal IDs; tags contain no sensitive traits.
- Least-privilege senders, MFA, approvals, and campaign audit logs.

Official implementation reference: OneSignal Flutter setup

12. Twilio Verify v2

Provider-managed phone/email OTP verification lifecycle, anti-fraud controls, and delivery across approved channels.

Priority	P0
Disposition	Refactor existing Twilio Messaging OTP to Verify v2
Current AMORAA state	twilio package and SMS adapter exist, but the backend generates/hashes OTPs locally and calls Programmable Messaging; no Verify Service SID.
Client dependency	Very High - Twilio account, identity/business verification, India messaging compliance/funding; 1-4 weeks.
Engineering size	M/L
Primary layer	Backend + existing Flutter OTP UI

> **Recommendation:** Use Verify v2 Verifications and VerificationCheck for phone verification. Retain local OtpTokens only for non-Verify email recovery/fallback if approved; do not maintain two authoritative phone-OTP lifecycles.

Implementation and connection to AMORAA

SDK/API: Twilio Node helper library; Verify v2 Services/Verifications/VerificationCheck APIs.

Flutter/web changes: Keep existing account verification UI but normalize E.164 numbers, surface provider-neutral resend/attempt errors, and add cooldown/accessibility behavior.

Node.js/Express changes: Replace createOtp/deliverOtp/verifyOtp for phone with Twilio Verify calls, map provider statuses/errors, rate limit by IP/phone/user, and use restricted API keys/subaccounts by environment.

MySQL requirements: Add VerificationAttempts/provider fields or repurpose OtpTokens for audit only (providerSid, channel, status, attempts, expiresAt); never store the provider OTP. Keep email reset tokens separate.

Configuration/environment: TWILIO_ACCOUNT_SID, TWILIO_API_KEY, TWILIO_API_SECRET (preferred), TWILIO_VERIFY_SERVICE_SID; optional messaging service/template configuration.

Webhooks/callbacks: Optional Verify status callbacks/fraud events; validate Twilio signatures and deduplicate.

Development team will implement

- Refactor phone OTP start/check endpoints to Verify v2 while preserving API response contracts.
- Implement rate limits, generic responses, provider error mapping, retry/backoff, and environment isolation.
- Add audit/metrics without storing codes.

- Prepare cutover/fallback/runbook and update tests/Postman collection.

Client must provide or complete

- Create a company-owned Twilio organization/subaccount, complete identity/business verification, fund it, and invite least-privilege team members.
- Create Verify Services for non-production/production and provide SIDs/credentials through a secret manager.
- Provide legal entity, registered address, tax/billing, authorized signatory, support contact, and India DLT/sender/template details if required.
- Approve OTP brand text, channels, country allow-list, language, fraud thresholds, resend/attempt policy, and monthly budget alerts.

Implementation sequence

63. Complete Twilio/compliance onboarding.
64. Create Verify services and restricted credentials.
65. Implement start/check adapter and database audit.
66. Run test-number and real-device cases.
67. Gradually cut over and monitor conversion/fraud/cost.

Testing and acceptance

- Approved, pending, expired, wrong, max-attempt, resend-cooldown, blocked-country, and provider-outage cases.
- E.164 normalization and India test devices/networks.
- Rate limiting and generic responses prevent enumeration/OTP abuse.
- No OTP is logged or stored in production.

Security and privacy controls

- Prefer API keys over the primary Auth Token and rotate credentials.
- Do not log codes, full phone numbers, or provider request bodies.
- Use Twilio request signature verification for callbacks and separate subaccounts by environment.

Official implementation reference: Twilio Verify v2

13. Google OAuth / Sign in with Google

Google social sign-in with backend ID-token verification and AMORAA JWT session issuance.

Priority	P0
Disposition	Complete existing partial integration
Current AMORAA state	Flutter google_sign_in and backend google-auth-library verification exist; GOOGLE_CLIENT_IDS is not configured and platform OAuth files/identifiers are incomplete.
Client dependency	High - Google Cloud OAuth ownership, app IDs/signing, consent screen; 2-10 client days.
Engineering size	M
Primary layer	Flutter/web + backend

> **Recommendation:** Keep the existing backend ID-token verification pattern, but finalize identifiers, current SDK configuration, account linking policy, nonce/state, and web Google Identity Services behavior before enabling production.

Implementation and connection to AMORAA

SDK/API: google_sign_in/latest compatible Flutter plugin; Google Identity Services; google-auth-library on Express.

Flutter/web changes: Configure Android/iOS/web client IDs, platform URL schemes, consent/cancel/error UX, and send ID token over HTTPS to /api/auth/google.

Node.js/Express changes: Verify issuer/audience/expiry/nonce where applicable, validate email verification, handle provider collisions/linking, log security events, and issue existing access/refresh tokens.

MySQL requirements: Existing Users.googleId/authProvider works; add ExternalIdentities (provider, providerSubject unique, userId, emailAtLink, linkedAt, revokedAt) for multi-provider/Apple-ready linking.

Configuration/environment: GOOGLE_CLIENT_IDS or per-platform GOOGLE_*_CLIENT_ID; web client secret only if an authorization-code flow is chosen; allowed origins/redirect URIs in Google Cloud.

Webhooks/callbacks: No routine sign-in webhook. Optional Cross Account Protection can be evaluated later.

Development team will implement

- Update Google sign-in configuration for Android/iOS/web and preserve backend token verification.
- Implement safe account linking/collision policy and ExternalIdentities migration.
- Add telemetry/error mapping, logout/revocation behavior, and provider test coverage.
- Document production OAuth consent-screen/credential ownership.

Client must provide or complete

- Provide a corporate Google Cloud/Firebase project and OAuth administrator access.
- Finalize Android package, iOS bundle, web domain, Android release SHA-1/SHA-256, and store identifiers.
- Configure/approve OAuth consent screen, app name/logo, support email, privacy/terms URLs, authorized domains/origins/redirects.
- Approve whether matching-email local accounts may be linked, blocked, or require re-authentication.

Implementation sequence

68. Finalize identifiers/signing.
69. Create platform OAuth clients and consent screen.
70. Configure Flutter/web and backend audiences.
71. Implement/linking migration and errors.
72. Test internal then production OAuth verification.

Testing and acceptance

- New user, existing Google user, local-email collision, deleted/deactivated account, cancellation, expired/wrong-audience token.
- Android release signing, iOS URL return, and web origin tests.
- Account switch/logout clears provider/session state.
- Backend rejects client-supplied subject/email without a valid ID token.

Security and privacy controls

- ID tokens travel only over HTTPS and are verified server-side; never trust profile fields alone.
- Client secrets stay server-side; OAuth console access uses MFA/least privilege.
- Require explicit secure linking rules to prevent account takeover.

Official implementation reference: Google backend authentication

14. Sign in with Apple

Apple social sign-in for iOS/web with privacy-preserving email relay and backend session issuance.

Priority	P0
Disposition	Implement before iOS social-login release
Current AMORAA state	Absent; no entitlement, Service ID, key, Apple callback, nonce, or ExternalIdentities table.
Client dependency	Very High - Apple Developer Account Holder/Admin and verified domain; 3-15 client days.
Engineering size	M/L
Primary layer	iOS Flutter + backend (+ web if offered)

> **Recommendation:** Implement alongside Google login before iOS release when social login is offered. Persist the Apple subject and first-return profile immediately; users may hide their email and Apple usually returns name/email only on first authorization.

Implementation and connection to AMORAA

SDK/API: sign_in_with_apple Flutter package/native AuthenticationServices; Apple OpenID Connect token endpoints/JWKS.

Flutter/web changes: Add Apple button per platform guidelines, random nonce/state, iOS capability/entitlement, cancellation/error UX, and send identity token/authorization code to backend.

Node.js/Express changes: Verify Apple JWS issuer/audience/nonce, exchange/revoke codes where required, support private relay email, link through ExternalIdentities, and issue AMORAA tokens.

MySQL requirements: ExternalIdentities plus Users email/relay flags; store Apple subject and authorization state, not raw identity tokens.

Configuration/environment: APPLE_TEAM_ID, APPLE_BUNDLE_ID/CLIENT_ID, APPLE_SERVICE_ID, APPLE_KEY_ID, APPLE_PRIVATE_KEY secret, APPLE_REDIRECT_URI.

Webhooks/callbacks: Apple server-to-server account events/revocation endpoint if configured; verify signed tokens and update link status.

Development team will implement

- Add iOS capability/entitlement and Flutter flow with nonce/state.
- Implement Apple token verification, code exchange/revocation, relay-email handling, and account linking.
- Add ExternalIdentities migration and error/audit events.
- Prepare App Review test instructions and deletion/revocation behavior.

Client must provide or complete

- Provide Apple Developer organization Account Holder/Admin access and App Store Connect access.
- Register final App ID/bundle, enable Sign in with Apple, create Service ID if web use is needed, and provide verified domain/return URL.
- Create .p8 key/Key ID/Team ID and transfer via an approved secret channel; approve rotation owner.
- Approve private relay support, account-linking policy, privacy/terms/support URLs, and App Review account/instructions.

Implementation sequence

73. Finalize Apple organization/app identifiers.
74. Create capability, Service ID, key, domains/redirects.
75. Implement Flutter and backend verification/linking.
76. Test sandbox/TestFlight and revocation.
77. Submit with App Review notes.

Testing and acceptance

- First login, repeat login without returned name, Hide My Email, cancellation, nonce mismatch, revoked credential, account collision.
- Physical-device/TestFlight return flow and relay email delivery.
- Deletion revokes provider tokens where required and invalidates AMORAA sessions.
- Wrong audience/issuer/expired JWS is rejected.

Security and privacy controls

- Private key stays in secrets manager and never enters the app/repository.
- Use nonce/state and server verification; do not trust client profile claims.
- Minimize relay-email exposure and document account-linking/recovery paths.

Official implementation reference: Sign in with Apple

15. Google Play Billing

Android digital subscription purchase, verification, acknowledgement, renewal/cancel lifecycle, and entitlement sync.

Priority	P0
Disposition	Implement; replace mobile Razorpay for digital subscriptions
Current AMORAA state	Absent. Current payment UI uses razorpay_flutter and MySQL subscriptions/payments; no Play products, purchase token fields, Developer API, or RTDN.
Client dependency	Very High - verified Play Console organization, merchant/payments profile, app signing/release track, products; 1-6 weeks.
Engineering size	XL
Primary layer	Android Flutter + backend + MySQL

> **Recommendation:** Use Google Play Billing for Android in-app digital membership. Keep Razorpay only for eligible web/physical/off-app use after policy/legal review. MySQL remains the unified entitlement source after server verification.

Implementation and connection to AMORAA

SDK/API: Flutter in_app_purchase (or approved maintained billing wrapper), Google Play Billing Library, Google Play Developer API, Cloud Pub/Sub RTDN.

Flutter/web changes: Query localized ProductDetails, launch purchase, handle pending/cancelled/completed results, send purchase token/product ID to backend, restore/query purchases, and never grant premium locally.

Node.js/Express changes: Verify token with Developer API, bind obfuscated account ID, acknowledge initial purchase, process RTDN via Pub/Sub push/pull, handle renewal, grace, hold, pause, cancel, expire, refund/void, and reconcile.

MySQL requirements: Extend SubscriptionPlans with playProductId/basePlanId/offerId; Payments/Subscriptions with purchaseTokenHash, orderId, linkedPurchaseTokenHash, acknowledgementState, region, provider timestamps; add StoreEvents unique messageId.

Configuration/environment: GOOGLE_PLAY_PACKAGE_NAME, service account/workload identity credentials, GCP project/topic/subscription, RTDN auth audience, product mapping; credentials server-only.

Webhooks/callbacks: Cloud Pub/Sub RTDN endpoint authenticated with OIDC. Each RTDN triggers a Developer API lookup; messages are idempotent by messageId.

Development team will implement

- Refactor membership checkout into platform-specific purchase adapters and one backend entitlement service.
- Implement Play verification/acknowledgement, RTDN ingestion, lifecycle mapping, reconciliation, and migration.
- Create products/base plans/offers and safe mapping to existing SubscriptionPlans.
- Add Play license-test, closed-track, refund/cancel, finance/support runbooks.

Client must provide or complete

- Provide verified Google Play Console organization owner/admin access, final package ID, app signing configuration, and a closed test release.
- Create/approve merchant payments profile, tax/bank/settlement details, subscriptions/base plans/offers, prices/countries, grace/hold policy, and tester accounts.
- Provide GCP project/Pub/Sub access and authorize service account/Developer API linkage.
- Provide Terms, Privacy, subscription disclosure, refund/cancellation/support policies and pricing approval.

Implementation sequence

78. Finalize package/signing/store organization and product catalog.
79. Implement Flutter purchase adapter and backend verification schema.
80. Configure RTDN/Pub/Sub and lifecycle reconciliation.
81. Run license/closed-track testing including pending/refund/cancel.
82. Migrate checkout and enable production gradually.

Testing and acceptance

- Successful, pending, cancelled, duplicate, already-owned, restore, upgrade/downgrade, renewal, grace, hold, expire, refund, and revoked cases.
- RTDN duplicate/out-of-order/authentication tests plus Developer API reconciliation.
- Entitlement is granted only after verified PURCHASED state and acknowledgement.
- Cross-account purchase-token replay is rejected.

Security and privacy controls

- Never trust client purchase result alone or store raw service credentials in app.
- Hash/encrypt purchase tokens and enforce unique ownership/idempotency.
- Authenticate RTDN, verify product/package/amount/state, and maintain immutable event audit.

Official implementation reference: Google Play backend integration

16. Apple StoreKit / App Store Server API

iOS digital subscriptions, signed transaction verification, renewals/cancellations/refunds, and unified entitlements.

Priority	P0
Disposition	Implement
Current AMORAA state	Absent; no IAP capability/products, transaction IDs, App Store keys, or server notification endpoint.
Client dependency	Very High - Apple Account Holder, paid agreements, banking/tax, products/review; 1-6 weeks.
Engineering size	XL
Primary layer	iOS Flutter + backend + MySQL

> **Recommendation:** Use StoreKit/App Store purchase flow for iOS membership and App Store Server Notifications V2. MySQL grants access only after verified signed transaction/server status.

Implementation and connection to AMORAA

SDK/API: Flutter in_app_purchase or approved StoreKit 2 bridge; App Store Server API library; signed JWS transaction verification; Notifications V2.

Flutter/web changes: Load localized products, purchase/restore, finish transactions after backend confirmation, show pending/cancelled states, and open subscription management.

Node.js/Express changes: Verify signed transaction/JWS and bundle/environment/product, query transaction/subscription history/status, process V2 notifications, map renewal/grace/billing retry/expire/refund/revoke, and reconcile.

MySQL requirements: SubscriptionPlans appleProductId/groupId; Payments/Subscriptions originalTransactionId, transactionId, webOrderLineItemId, signedDate, environment, ownershipType; StoreEvents unique notificationUUID.

Configuration/environment: APPLE_ISSUER_ID, APPLE_KEY_ID, APPLE_IAP_PRIVATE_KEY, APPLE_BUNDLE_ID, APPLE_APP_ID, environment URLs, product mapping.

Webhooks/callbacks: HTTPS App Store Server Notifications V2 for sandbox/production; verify signedPayload/JWS and notificationUUID idempotency; support test-notification API.

Development team will implement

- Implement platform purchase adapter and unified entitlement mapping.
- Build JWS verification, Server API client, V2 notification ingestion, reconciliation, and support lookup.
- Configure products/subscription group/offers and map to MySQL plans.
- Add StoreKit local, sandbox, TestFlight, refund/revoke, and restore runbooks.

Client must provide or complete

- Provide Apple Developer/App Store Connect Account Holder/Admin access and final bundle/App ID.
- Accept paid-app agreements and provide legal entity, banking, tax, pricing/country, subscription group/products/offers, localization, and review metadata.
- Create App Store Server API key (.p8), Issuer ID, Key ID and share securely; configure notification URLs.
- Provide subscription disclosures, privacy/terms, refund/support contacts, and App Review credentials/instructions.

Implementation sequence

83. Complete agreements/store product setup.
84. Implement Flutter StoreKit purchase/restore and backend schema.
85. Implement signed transaction verification and Server API.
86. Configure/test Notifications V2 sandbox/TestFlight.
87. Enable production and daily reconciliation.

Testing and acceptance

- StoreKit local, sandbox, TestFlight purchase/restore/cancel/renew/billing retry/grace/refund/revoke/upgrade/downgrade.
- V2 TEST notification, duplicates, wrong bundle/environment, invalid JWS, and out-of-order events.
- Entitlement follows verified original transaction lifecycle across devices.
- Account switch cannot claim another user's transaction.

Security and privacy controls

- .p8 key is server-only with rotation/owner record.
- Verify Apple certificate/JWS chain, bundle, environment, product, and dates.
- Store minimal transaction identifiers and immutable event hashes, not receipts in logs.

Official implementation reference: App Store Server API

17. Razorpay (Web payments/subscriptions)

Eligible web checkout and INR payment/subscription processing outside mandatory mobile-store billing scope.

Priority	P1
Disposition	Harden and reposition existing integration
Current AMORAA state	Flutter mobile checkout, backend order/signature/payment fetch, webhook HMAC, Payments/PaymentEvents/Subscriptions already exist; credentials absent.
Client dependency	Very High - Razorpay KYC/activation, bank/tax/domain/policies; 1-4 weeks.
Engineering size	M/L
Primary layer	Flutter Web/web checkout + backend

> **Recommendation:** Retain the strong existing backend verification/idempotency path, but limit production Razorpay use to policy-eligible web/physical/off-app cases. Do not use it to bypass Play/App Store billing for in-app digital membership.

Implementation and connection to AMORAA

SDK/API: Razorpay Orders/Payments/Subscriptions APIs and Web Checkout; existing razorpay_flutter only where platform policy permits.

Flutter/web changes: Use web checkout/hosted flow with server-created order and existing success/failure/pending UI; never embed key secret. Add return/recovery polling for interrupted checkout.

Node.js/Express changes: Keep order creation, HMAC checkout verification, provider payment fetch, raw-body webhook validation, event dedupe; add refunds/subscription lifecycle and environment/merchant mapping as selected.

MySQL requirements: Existing Payments, PaymentEvents, Subscriptions are suitable; add providerSubscriptionId, invoiceId, refund/dispute details, checkoutPlatform, tax/receipt references and reconciliation status.

Configuration/environment: Existing RAZORPAY_KEY_ID, KEY_SECRET, WEBHOOK_SECRET, API_BASE_URL; separate test/live secrets and webhook URLs.

Webhooks/callbacks: payment.captured/failed, order.paid, refund, dispute/chargeback, subscription/invoice events if Razorpay Subscriptions is used; raw body + signature + event ID dedupe.

Development team will implement

- Complete web checkout adapter and remove/disable non-compliant mobile digital checkout paths.
- Expand webhook lifecycle/refund/reconciliation and preserve existing idempotency/verification.
- Add policy-based product/channel gating, finance exports, receipts, and support tools.
- Update integration tests using test mode and production go-live checklist.

Client must provide or complete

- Create/activate company Razorpay account and complete KYC with legal entity, PAN/GST/CIN as applicable, registered address, website/domain, business proof, bank account/cancelled cheque, authorized signatory, and settlement details.
- Provide owner/admin/developer access, test/live keys via secret manager, webhook secret, and approved webhook domain.
- Approve eligible products/platforms, prices/taxes/refunds/cancellations, receipt/invoice text, and settlement/reconciliation owner.
- Provide Terms, Privacy, Refund/Cancellation Policy, Contact/Support page, and billing contact.

Implementation sequence

88. Complete activation/KYC and policy scope.
89. Configure test keys/webhook and web checkout.
90. Harden lifecycle/refund/reconciliation and channel gating.
91. Run test-mode payment/refund/dispute cases.
92. Rotate to live keys and monitor settlements.

Testing and acceptance

- Success, failure, cancel, pending/external wallet, duplicate order/idempotency, invalid signature, amount mismatch, refund/dispute, and webhook replay/out-of-order.
- Test-mode dashboard/payment records reconcile to MySQL.
- Live smoke payment/refund uses client-approved minimal amount and finance sign-off.
- Mobile digital products cannot select Razorpay when store billing is required.

Security and privacy controls

- Secrets are backend-only; checkout key ID is the only public credential.
- Use raw-body HMAC and unique event IDs as the current code already does.
- Avoid storing card/payment instrument data; rely on Razorpay-hosted checkout and audit access.

Official implementation reference: Razorpay webhook validation

18. Google Maps Platform

Maps, location selection, place autocomplete/details, distance, and venue/date-spot discovery.

Priority	P0
Disposition	Implement
Current AMORAA state	Android location permissions exist; DateSpotsMapScreen explicitly says map unavailable and uses static venue data. No Maps SDK/package/API keys/place schema.
Client dependency	Very High - Google Cloud billing, final app IDs/signing/domain, budget; 2-10 client days.
Engineering size	L
Primary layer	Flutter/web + backend + MySQL

> **Recommendation:** Use platform-restricted keys for map rendering and a backend-restricted key/proxy for Places/Routes data where business logic or secret controls are needed. Store Google place_id plus selected business fields, not copied provider data indefinitely beyond terms.

Implementation and connection to AMORAA

SDK/API: google_maps_flutter, Maps JavaScript API for web if supported, Places API (New), Geocoding/Routes/Distance Matrix equivalent as approved.

Flutter/web changes: Replace map preview with live map, permission rationale, approximate-location fallback, autocomplete session tokens, selected place details, markers, and graceful quota/network fallback.

Node.js/Express changes: Provide venue/place search proxy if needed, restrict fields, cache only permitted data, compute/store internal distance/location indexes, and enforce quotas/rate limits.

MySQL requirements: Add VenueLocations/Places (googlePlaceId unique, name/address, lat/lng POINT or decimals, categories, source, lastRefreshedAt) and Event placeId/lat/lng references; add spatial indexes if query volume warrants.

Configuration/environment: MAPS_ANDROID_API_KEY, MAPS_IOS_API_KEY, MAPS_WEB_API_KEY public but restricted; MAPS_SERVER_API_KEY server-only; GOOGLE_MAP_ID; API allow-lists and budget alerts.

Webhooks/callbacks: None for standard Maps/Places. Scheduled refresh may be used within provider terms.

Development team will implement

- Enable only required APIs and create per-platform restricted keys.
- Implement map/autocomplete/place details with session tokens and fallback UX.
- Add venue/location schema, geospatial query strategy, rate limits, caching, and cost telemetry.

- Test permission, key restrictions, quota, billing, and accessibility.

Client must provide or complete

- Provide client-owned Google Cloud project, billing profile/payment method, Maps admin/billing access, and monthly budget/alert thresholds.
- Provide final package/bundle/domain, Android release signing fingerprints, iOS bundle restrictions, and production web referrers.
- Approve location consent wording, approximate/precise use, retention, venue curation, countries/cities, and cost ceiling.
- Provide production/staging domains and legal/privacy policy coverage for location data.

Implementation sequence

93. Finalize identifiers/billing and enable minimal APIs.
94. Create restricted keys and map IDs/styles.
95. Implement Flutter UI and backend/schema.
96. Run quota/cost/security tests.
97. Launch city-by-city with monitoring.

Testing and acceptance

- Physical-device permissions: denied, approximate, precise, revoked, location off.
- Autocomplete session, place details, map tiles, route/distance, invalid/expired key, and quota failures.
- API key restriction tests from unauthorized app/domain/server.
- Venue coordinates/distance and privacy deletion behavior are verified.

Security and privacy controls

- Restrict every key by API and application; server key never ships in app.
- Do not expose exact user location to other users or marketing tools.
- Use minimum precision/retention and explicit location-purpose consent.

Official implementation reference: Places Autocomplete (New)

19. Cloudinary

Profile/event/chat media storage, resizing, optimization, delivery, and optional moderation workflow.

Priority	P0
Disposition	Implement for public/non-regulated media
Current AMORAA state	Profile photos are written under public /uploads; chat/KYC media use local private disk. No durable object storage/CDN/media asset model.
Client dependency	High - vendor plan/billing, media policy, domain; 2-10 client days.
Engineering size	L
Primary layer	Flutter/web + backend + MySQL

> **Recommendation:** Use signed backend-mediated uploads for profile/event/chat media and store asset IDs, not only URLs. Do not place raw Aadhaar/KYC images in public delivery; use private/authenticated assets or the separately approved KYC/S3 path.

Implementation and connection to AMORAA

SDK/API: Cloudinary Node SDK/Upload API; cloudinary_flutter for delivery; signed upload parameters; webhooks/moderation add-ons if selected.

Flutter/web changes: Request upload signature/session or upload through Express, show compression/progress/retry, then render approved transformations with fallback and placeholder.

Node.js/Express changes: Validate magic bytes/size/ownership, sign/upload, set folders/tags/context/access type, receive moderation callbacks, delete/replace old assets, and issue signed URLs for private media.

MySQL requirements: Create MediaAssets (id, userId, provider, publicId, resourceType, version, secureUrl, accessType, purpose, bytes, dimensions, moderationStatus, checksum, deletedAt); migrate OnboardingProfiles.photos and MessageMedia references.

Configuration/environment: CLOUDINARY_CLOUD_NAME and API_KEY; CLOUDINARY_API_SECRET server-only; upload preset/transformation/moderation webhook secret; CDN/custom-domain settings.

Webhooks/callbacks: Upload/moderation/analysis callbacks if enabled; verify signature/timestamp and map by public_id/version.

Development team will implement

- Design MediaAssets schema and purpose-based access policy.
- Implement signed uploads, transformations, private delivery, deletion, migration, and provider abstraction.
- Integrate moderation state without exposing unapproved media.

- Add lifecycle cleanup, backup/export, cost/quota alerts, and media QA.

Client must provide or complete

- Create a company Cloudinary product environment/organization, choose plan/region, add billing, and invite roles.
- Provide cloud name/account access and approve signed-upload/private delivery architecture.
- Approve media limits, formats, transformations, retention/deletion, moderation provider/thresholds, and acceptable-use policy.
- Provide custom media domain/DNS access if desired and legal/privacy consent for media processing.

Implementation sequence

98. Approve media classification/provider plan.
99. Create MediaAssets schema and provider adapter.
100. Implement signed upload/delivery/moderation/deletion.
101. Migrate local development paths in staging.
102. Load/security/cost test then cut over.

Testing and acceptance

- Valid/invalid MIME and magic bytes, oversize, multi-upload, retry, duplicate/checksum, replace/delete, authorization, signed URL expiry.
- Transformation quality/orientation/format and slow-network behavior.
- Moderation pending/approved/rejected callback and replay cases.
- No KYC/private chat asset is accidentally publicly addressable.

Security and privacy controls

- API secret is backend-only; prefer signed uploads and restricted presets.
- Use authenticated/private delivery for sensitive media and short-lived URLs.
- Strip metadata as approved, malware/content-scan uploads, and audit deletions/access.

Official implementation reference: Cloudinary Upload API

20. AWS S3 + CloudFront (alternative)

Infrastructure-controlled object storage/CDN with private buckets, presigned uploads, lifecycle policies, and signed delivery.

Priority	P1
Disposition	Alternative/complement for regulated private media
Current AMORAA state	Absent; local disk is not horizontally durable and no cloud IAM/bucket/CDN exists.
Client dependency	Very High - AWS organization/billing/IAM/DNS/security ownership; 1-3 weeks.
Engineering size	XL
Primary layer	Backend/infrastructure + Flutter upload client

> **Recommendation:** Choose S3/CloudFront instead of Cloudinary for the full media stack, or use it narrowly for KYC/private evidence when infrastructure control is required. Do not run duplicate public-media systems without an explicit storage-classification decision.

Implementation and connection to AMORAA

SDK/API: AWS SDK for JavaScript v3 (S3, CloudFront); presigned URL flow; CloudFront Origin Access Control and signed URLs/cookies.

Flutter/web changes: Upload via short-lived presigned PUT/multipart instructions with checksum, progress, retry, and completion callback; consume short-lived signed delivery URLs.

Node.js/Express changes: Authorize object key/purpose, generate presigned upload/download, verify size/checksum/content type after upload, trigger scanning/processing, delete/lifecycle, and sign CloudFront URLs.

MySQL requirements: MediaAssets with bucket/key/version/checksum/access class; UploadSessions; optional scan/moderation state. Never make S3 object keys the sole business record.

Configuration/environment: AWS_REGION, S3_* bucket names, CLOUDFRONT_DISTRIBUTION/KEY_PAIR, KMS key IDs; prefer workload roles over long-lived AWS keys.

Webhooks/callbacks: S3 Event Notifications to SQS/Lambda/HTTPS worker for scan/process; idempotent by event/object version.

Development team will implement

- Produce Cloudinary-vs-AWS storage decision and data classification.
- Provision private buckets, OAC CloudFront, IAM roles/KMS/lifecycle/logging/backup.
- Implement upload sessions, checksums, scan pipeline, signed delivery, and MediaAssets mapping.
- Add infrastructure-as-code, disaster recovery, cost alerts, and runbook.

Client must provide or complete

- Provide client-owned AWS Organization/account, billing/payment, preferred region, security/IAM administrator, and budget.
- Approve data residency, encryption/KMS ownership, retention/legal hold, backup/DR, CDN custom domain, and media classification.
- Provide DNS/certificate access and security/compliance requirements.
- Nominate operations owner for AWS cost, incidents, keys, and access reviews.

Implementation sequence

103. Approve architecture/classification/region.
104. Provision infrastructure as code and least-privilege roles.
105. Implement MediaAssets/upload/scan/signed delivery.
106. Migrate/test staging data.
107. Security/load/DR test then production cutover.

Testing and acceptance

- Unauthorized bucket/object access fails; presigned URL method/key/expiry/checksum limits are enforced.
- Multipart retry, corrupted checksum, oversized file, malware/quarantine, delete/lifecycle, and versioning.
- CloudFront OAC blocks direct public S3 access; signed URLs expire.
- Backup/restore, cross-zone failure, and cost alarms are exercised.

Security and privacy controls

- Private-by-default buckets, Block Public Access, OAC, KMS, least-privilege roles, CloudTrail/access logging.
- No long-lived AWS access keys in Flutter or repository.
- Separate KYC/private/public buckets and lifecycle policies.

Official implementation reference: S3 presigned URLs

21. Stream Chat OR Sendbird

Managed real-time chat, delivery/read state, presence, moderation hooks, and scalable messaging operations.

Priority	P0
Disposition	Architectural decision - replace existing chat or retain it
Current AMORAA state	AMORAA already has authenticated Socket.IO chat, MySQL conversations/messages/media/read state/presence/mute and tests. No managed chat SDK.
Client dependency	Very High - vendor selection/contract/data residency/migration; 2-6 weeks.
Engineering size	XL
Primary layer	Flutter + backend + managed chat

> **Recommendation:** Do not layer a managed provider on top of current Socket.IO. Choose either (a) retain/harden the working in-house chat, or (b) migrate to one vendor with a defined cutover, history, moderation, and source-of-truth plan. Stream is a natural Flutter candidate; Sendbird remains a viable alternative.

Implementation and connection to AMORAA

SDK/API: stream_chat_flutter + server SDK/webhooks, or sendbird_chat_sdk + Platform API/webhooks.

Flutter/web changes: Replace ChatRepository with provider adapter, connect using short-lived backend-generated token, map users/channels/messages/read/presence/typing/media, and preserve current UI/access rules.

Node.js/Express changes: Generate expiring provider tokens, synchronize user/channel lifecycle after active match/block/delete, verify webhooks, enforce bans/moderation, and maintain only required shadow metadata.

MySQL requirements: Decide system of record. Add ChatProviderMappings and ProviderEvents; either migrate/archive existing Messages or keep an immutable compliance/audit projection without dual-writes indefinitely.

Configuration/environment: CHAT_PROVIDER, STREAM_API_KEY/SECRET or SENDBIRD_APP_ID/API_TOKEN, webhook secret, region, moderation config; secret server-only.

Webhooks/callbacks: Message/channel/user/moderation delivery events; verify signatures, deduplicate, and handle retries/outages.

Development team will implement

- Prepare build-vs-buy decision including current chat evidence, scale/SLA, moderation, data residency, pricing, and migration cost.

- If selected, build provider abstraction, backend token endpoint, lifecycle/webhook handlers, and migration tooling.
- Preserve block/match/account deletion authorization and media policy.
- Run dual-read/cutover strategy without long-term duplicate sending.

Client must provide or complete

- Approve retain-in-house versus Stream/Sendbird and, if vendor, select commercial plan/region/SLA/DPA with billing and owner access.
- Approve chat retention, moderation rules, escalation, data export/deletion, support access, and prohibited-content policy.
- Provide expected MAU/concurrency/message/media volumes and incident/support requirements.
- Nominate trust-and-safety, legal/privacy, support, and procurement owners.

Implementation sequence

108. Decision workshop and vendor proof of concept.
109. Approve data model/migration/source of truth.
110. Implement token/lifecycle/webhook/provider repository.
111. Migrate pilot cohort and validate moderation/history.
112. Cut over with rollback and reconciliation.

Testing and acceptance

- Only active matched users can connect/create/read; block/unmatch/delete revokes access quickly.
- Offline/reconnect/order/read/presence/typing/media/push/webhook duplicate and provider outage cases.
- History migration checksums/counts and user isolation.
- Load, rate-limit, moderation, deletion/export, and rollback tests.

Security and privacy controls

- Provider secret/API token stays backend-only; use short-lived user tokens.
- Verify webhook signatures and re-authorize lifecycle events against AMORAA state.
- Encrypt/minimize chat data, restrict support access, and define retention/legal hold.

Official implementation reference: Stream Flutter go-live checklist

22. Identity / KYC provider

ID document and selfie/liveness verification with auditable provider results and minimal regulated-data exposure.

Priority	P0
Disposition	Provider/legal decision required before production KYC
Current AMORAA state	AMORAA accepts Aadhaar+selfie into private local storage and records pending status; no provider, liveness/OCR, admin review, or external decision callback.
Client dependency	Critical - legal review, vendor contract/KYB, approved Aadhaar method and data flow; 2-8+ weeks.
Engineering size	XL
Primary layer	Flutter + backend + MySQL + KYC provider

> **Recommendation:** Pause production use of raw Aadhaar-image capture until the client selects a legally approved provider/method and counsel confirms purpose, consent, storage, masking, retention, and UIDAI/DPDP compliance. Prefer provider-hosted capture/tokenized results; never use Aadhaar number as AMORAA's user identifier.

Implementation and connection to AMORAA

SDK/API: Selected India-capable KYC provider SDK/hosted flow/API for document OCR, face match/liveness and permitted Aadhaar/offline/DigiLocker flow; signed webhooks.

Flutter/web changes: Replace direct upload with provider session/SDK or hosted flow, explicit consent/purpose notice, camera/liveness UX, retry/manual fallback, and status screen.

Node.js/Express changes: Create provider session, bind user/reference, verify webhook signature/status, map normalized result to pending/verified/rejected/manual_review, fetch only necessary fields, and enforce review/admin authorization.

MySQL requirements: Extend IdentityVerifications with provider, providerReference unique, method, consentVersion/time, maskedDocumentLast4 or token only, liveness/face result summary, decision codes, expiresAt; remove/expire raw document paths per approved policy and add immutable audit events.

Configuration/environment: KYC_PROVIDER, client/account IDs, API keys/certificates, webhook secret, encryption key, region/base URLs; no KYC secrets or raw responses in Flutter/logs.

Webhooks/callbacks: Signed verification.completed/failed/manual-review callbacks with timestamp/replay protection/idempotency; poll only as fallback.

Development team will implement

- Run vendor/legal architecture review and data-protection impact assessment before coding production flow.

- Implement provider session, Flutter capture/hosted flow, signed webhook, status mapping, and minimal schema.
- Implement admin/manual-review boundaries, retention/deletion, redaction, access logging, and incident response.
- Migrate or purge current raw development submissions according to approved policy.

Client must provide or complete

- Select/contract the provider and complete its KYB: legal entity, CIN/GST/PAN as applicable, registered address, authorized signatory, directors/beneficial ownership, bank/billing, website/app, use-case volumes, security/privacy contacts, and any provider/UIDAI eligibility documents.
- Obtain written legal advice approving the exact Aadhaar/offline/DigiLocker/document flow, purpose, consent, retention, cross-border processing, age policy, and grievance/DPO contact.
- Provide client-owned provider account/admin access, sandbox/live credentials/certificates via secret manager, webhook allow-list, and production approval.
- Approve accepted documents, retry/manual-review/rejection policy, retention/deletion timetable, support copy, and escalation SLA.

Implementation sequence

113. Legal/DPIA and provider selection/KYB.
114. Approve data-flow diagram and retention/minimization.
115. Implement sandbox session/capture/webhook/status.
116. Security/privacy/manual-review penetration and operational testing.
117. Limited pilot then production approval; purge legacy raw files as directed.

Testing and acceptance

- Happy path, document mismatch, liveness fail, poor image, duplicate session, timeout, user cancellation, provider outage, manual review, rejection/resubmission.
- Forged/replayed/out-of-order webhook and cross-user reference tests.
- Access logs, encryption, retention expiry, deletion, data export/redaction, and incident drill.
- No raw Aadhaar number/biometric/OTP appears in logs, analytics, support, or marketing systems.

Security and privacy controls

- UIDAI states Aadhaar biometric/OTP authentication data must not be stored permanently and Aadhaar must not be a domain identifier; counsel/provider must confirm the selected flow.
- Prefer tokenized/masked results and provider-hosted capture; segregate KYC data and keys from general media.
- Strict role-based access, encryption, audit, redaction, retention, and breach-response controls.

Official implementation reference: UIDAI requesting-entity security requirements

23. Sentry (optional)

Cross-stack exception/performance monitoring, release correlation, traces, and backend operational diagnostics.

Priority	P1
Disposition	Optional; recommended primarily for Express backend
Current AMORAA state	Absent. Flutter has no Crashlytics/Sentry; Express logs to console only and has no release-aware APM.
Client dependency	Medium - organization/project/billing/data governance; 1-3 client days.
Engineering size	M
Primary layer	Flutter + Node/Express

> **Recommendation:** Use Firebase Crashlytics as P0 Flutter crash reporting and Sentry for Node/Express APM. Add Sentry Flutter only if the client explicitly wants one cross-stack tool and accepts overlap/cost with Crashlytics.

Implementation and connection to AMORAA

SDK/API: @sentry/node for Express; sentry_flutter optional; source-map/symbol upload CLI/integration.

Flutter/web changes: If enabled, initialize before app start, scrub context, set release/environment/user ID, capture unhandled errors/traces, and upload symbols without PII.

Node.js/Express changes: Initialize before Express routes, add tracing/error middleware in correct order, tag request/user/release, scrub bodies/headers, and exclude expected business errors.

MySQL requirements: No Sentry business table; optional IntegrationIncident references for operational workflows. Do not store DSN/auth token in MySQL.

Configuration/environment: SENTRY_DSN, SENTRY_ENVIRONMENT, SENTRY_RELEASE, traces/profile sample rates; SENTRY_AUTH_TOKEN only in CI for symbol/source-map upload.

Webhooks/callbacks: Optional issue alert webhook to support/incident tooling; verify secret and avoid exposing event payloads.

Development team will implement

- Create backend Sentry project and privacy scrubber/allow-list.
- Instrument Express errors, performance, release/deploy, and health alerts.
- Optionally instrument Flutter if duplication is approved.
- Configure symbol/source-map upload and incident runbook.

Client must provide or complete

- Approve Sentry use/plan/region/DPA and create client-owned organization/projects.

- Invite development/operations roles and provide DSNs; provide CI token securely if symbol upload is approved.
- Approve retention, sampling, alert recipients, on-call workflow, and data scrubbing.
- Nominate production incident owner.

Implementation sequence

118. Approve scope (backend only vs full stack).
119. Create projects/roles and scrub rules.
120. Instrument non-production and release pipeline.
121. Generate controlled errors/performance traces.
122. Enable production sampling/alerts.

Testing and acceptance

- Controlled backend exception and slow transaction appear with correct release/environment.
- Request bodies, Authorization, cookies, OTP, KYC, messages, email/phone are absent/redacted.
- Source maps/symbols resolve stack traces.
- Alert routing and quota/sampling behavior are exercised.

Security and privacy controls

- DSN is not treated as authorization; CI auth token is secret and scoped.
- Use deny-before-send/allow-list scrubbing and low sampling for sensitive routes.
- Restrict issue access and set appropriate retention/deletion.

Official implementation reference: Sentry Flutter package

24. Firebase Crashlytics

Production Flutter crash, non-fatal, and Android ANR visibility with release/symbol mapping.

Priority	P0
Disposition	Implement
Current AMORAA state	Absent; main() does not initialize Firebase or install fatal/asynchronous error handlers.
Client dependency	High - Firebase access and production signing/release pipeline; 1-3 client days.
Engineering size	M
Primary layer	Flutter Android/iOS

> **Recommendation:** Make Crashlytics the canonical mobile crash tool. Do not attach sensitive user/profile/KYC/chat content to custom keys or logs.

Implementation and connection to AMORAA

SDK/API: firebase_crashlytics + firebase_core; Crashlytics Gradle/Xcode build phases and symbol upload.

Flutter/web changes: Initialize Firebase, wire FlutterError.onError and PlatformDispatcher.onError, set release-safe user identifier and bounded non-sensitive keys, opt-in/collection behavior, and controlled non-fatal capture.

Node.js/Express changes: None. Backend APM is handled separately by Sentry or another server tool.

MySQL requirements: None; optionally record client app version/build in support tickets, not raw crash reports.

Configuration/environment: Firebase app config per environment; FIREBASE_APP_ID for symbol upload; CI service identity/CLI auth.

Webhooks/callbacks: Optional alert integrations from Firebase/Google Cloud; no application webhook required.

Development team will implement

- Install/configure Crashlytics and top-level error handlers.
- Configure build IDs, obfuscation/symbol upload, release/environment tagging, and privacy scrubbing.
- Add controlled crash/non-fatal QA and alert routing.
- Document triage, ownership, severity, and release regression workflow.

Client must provide or complete

- Provide Firebase project owner/admin access and production app registrations.
- Provide release signing/store/CI access needed to create builds and upload symbols.
- Approve crash collection/opt-out, retention, alert recipients, and privacy disclosure.

- Nominate mobile crash triage and release owners.

Implementation sequence

123. Configure Firebase apps and plugin/build scripts.
124. Add handlers/keys/privacy controls.
125. Upload symbols for test release.
126. Force test crash/non-fatal on physical devices.
127. Enable production alerts and release process.

Testing and acceptance

- Forced fatal/non-fatal on Android/iOS appears with correct build and readable symbols.
- Background/async Flutter errors are captured.
- User ID is pseudonymous and cleared/changed on account switch.
- No PII/sensitive values are attached to reports.

Security and privacy controls

- Use only pseudonymous user ID and allow-listed keys.
- Protect Firebase roles/CI tokens and separate environments.
- Document collection consent/retention and account-deletion expectations.

Official implementation reference: Crashlytics for Flutter

25. Firebase App Check

Attestation that requests originate from genuine AMORAA app instances and reduction of scripted/API abuse.

Priority	P0
Disposition	Implement after final signing/app IDs
Current AMORAA state	Absent. Express accepts JWT-authenticated traffic without app attestation; Firebase resources/config are absent.
Client dependency	High - Firebase/Play/Apple app setup and production signing; 2-10 client days.
Engineering size	L
Primary layer	Flutter + custom Express backend

> **Recommendation:** Use App Check as defense in depth, not as user authentication. Start in monitoring mode, verify tokens on high-risk Express routes, then enforce gradually with documented web/debug behavior and emergency bypass.

Implementation and connection to AMORAA

SDK/API: firebase_app_check; Play Integrity (Android), App Attest/DeviceCheck (Apple), reCAPTCHA provider for web; Firebase Admin SDK token verification on custom backend.

Flutter/web changes: Activate provider after Firebase initialization, fetch/refresh App Check token, attach X-Firebase-AppCheck to API requests, and handle attestation failure without blocking account recovery indiscriminately.

Node.js/Express changes: Verify App Check token/project/app ID before selected routes, cache JWKS/admin verification, log only result/fingerprint, rate limit, and support monitor/enforce/kill-switch modes.

MySQL requirements: No token storage. Add SecurityEvent/abuse counters if not already present; store verification outcome/app ID, not raw token.

Configuration/environment: FIREBASE_PROJECT_ID/admin credentials; APP_CHECK_MODE monitor|enforce; allowed Firebase app IDs; debug tokens only in non-production secrets.

Webhooks/callbacks: None. App Check token verification is request-time.

Development team will implement

- Configure providers for dev/staging/prod and Flutter token header injection.
- Add Express verification middleware and route risk tiers.
- Run monitoring to measure false rejects, then staged enforcement.
- Add abuse dashboards, debug/test strategy, emergency rollback, and documentation.

Client must provide or complete

- Provide Firebase/GCP owner access, final package/bundle/web domain, Play Console/Apple Developer access, and release signing.
- Approve which routes require enforcement, rollout risk, supported device/OS policy, and customer-support fallback.
- Provide physical test devices including older/non-standard devices within support policy.
- Nominate security/operations owner for false-positive review and emergency disable.

Implementation sequence

128. Finalize IDs/signing and configure attestation providers.
129. Implement Flutter token transport and backend monitor middleware.
130. Analyze metrics and fix compatibility gaps.
131. Enforce high-risk routes, then broaden.
132. Run quarterly keys/provider review.

Testing and acceptance

- Valid release, debug/staging, emulator/simulator, rooted/jailbroken or unsupported, replayed/expired/wrong-app token.
- JWT without App Check is monitored/rejected according to route/mode.
- Outage/kill switch and clock/JWKS refresh behavior.
- Web provider and accessibility/account-recovery flows do not deadlock users.

Security and privacy controls

- App Check does not replace JWT, authorization, rate limits, or server validation.
- Never log raw attestation tokens or ship debug tokens in release builds.
- Enforce expected project/app IDs and least-privilege Firebase admin access.

Official implementation reference: Firebase App Check Flutter setup

26. Firebase Remote Config

Remote feature flags, controlled rollouts, kill switches, copy/configuration, and experiments.

Priority	P1
Disposition	Implement after analytics/governance
Current AMORAA state	Absent; features and thresholds are compiled into Flutter/backend code.
Client dependency	Medium - Firebase roles and product approval/flag owners; 1-3 client days.
Engineering size	M
Primary layer	Flutter + optional backend mirror

> **Recommendation:** Use for non-secret presentation/behavior flags only. Authorization, price, entitlement, KYC, security limits, and provider secrets remain server-authoritative.

Implementation and connection to AMORAA

SDK/API: firebase_remote_config; optional Remote Config REST/Admin access for release automation.

Flutter/web changes: Set safe in-code defaults, fetch/activate after consent/network policy, typed flag registry, minimum fetch interval by environment, exposure events, and graceful offline fallback.

Node.js/Express changes: Optionally mirror a small approved flag set through server config; never trust client Remote Config for access control.

MySQL requirements: FlagExposure or ExperimentAssignment only if needed for server reconciliation; otherwise analytics event outbox captures exposure. No secrets.

Configuration/environment: Firebase config and REMOTE_CONFIG_MIN_FETCH_INTERVAL; CI credentials if templates are versioned/deployed.

Webhooks/callbacks: None for standard use. CI publishes versioned templates with review/rollback.

Development team will implement

- Define typed flag registry, owners, default/fail-safe values, expiry dates, and naming.
- Implement fetch/activate/exposure and offline/error behavior.
- Create review/publish/rollback process and template export in source control.
- Add guardrails preventing security/price/entitlement decisions in client flags.

Client must provide or complete

- Provide Firebase Remote Config access and nominate product/engineering publishers/approvers.
- Approve initial flags, targeting attributes, rollout cohorts, experiment metrics, and privacy constraints.
- Approve emergency kill-switch owners and after-hours escalation.

- Provide release calendar and business acceptance criteria.

Implementation sequence

133. Establish governance and defaults.
134. Implement typed service and exposure events.
135. Create non-production template/flags.
136. Test staged percentage/audience rollout and rollback.
137. Promote to production with expiry review.

Testing and acceptance

- Defaults before fetch/offline, fetch failure, throttling, stale config, invalid type, rollback.
- Targeting and exposure event match expected cohort without sensitive attributes.
- Security/entitlement remains enforced when a flag is tampered with.
- Kill switch takes effect within documented fetch behavior.

Security and privacy controls

- All Remote Config values are public to app users; never store secrets.
- Do not target by sensitive dating/KYC traits.
- Use least-privilege publishing, review, audit, and flag expiry.

Official implementation reference: Firebase Remote Config for Flutter

27. OpenAI API / approved LLM provider

Roadmap-dependent assistant, matching explanations, icebreakers, bio/prompt analysis, and safety-aware AI features.

Priority	P1
Disposition	Decision gate; backend-only implementation
Current AMORAA state	No provider SDK/API. Active matching explanation is deterministic; AI coach/icebreaker UI exists but must not claim external AI results without integration.
Client dependency	Very High - approved use cases, budget, safety/privacy/legal review, vendor account; 1-3 weeks.
Engineering size	XL
Primary layer	Backend + Flutter UI + MySQL audit

> **Recommendation:** Start with one narrow, low-risk, user-initiated use case (for example, profile-based icebreakers) and a non-AI fallback. Never send KYC, raw private messages, precise location, contact details, or full profile history; require product/legal approval before automated match decisions.

Implementation and connection to AMORAA

SDK/API: Official OpenAI Node SDK and Responses API (or approved equivalent), moderation/safety controls, structured outputs, store:false where appropriate.

Flutter/web changes: Call AMORAA backend only, show loading/cancel/retry/disclaimer, allow report/feedback, preserve deterministic fallback, and avoid representing generated text as objective compatibility.

Node.js/Express changes: Authorize/rate limit, assemble minimized/redacted prompt, version templates, request structured output, validate schema/content, apply safety filters, cache if allowed, record cost/latency/outcome, and provide circuit breaker/fallback.

MySQL requirements: AiRequests (userId, useCase, promptVersion, model, inputHash, outputHash or approved text, safetyStatus, tokens/cost, latency, providerRequestId, consentVersion, createdAt) and AiFeedback; no raw KYC/messages by default.

Configuration/environment: OPENAI_API_KEY, OPENAI_PROJECT_ID/ORG if used, model IDs, prompt version, timeouts, budgets, store flag; key server-only in secret manager.

Webhooks/callbacks: Usually none for synchronous Responses; background/batch callbacks/polling only if selected and compatible with retention policy.

Development team will implement

- Define approved use case, data-flow/privacy/safety threat model, quality rubric, and non-AI fallback.

- Implement backend-only provider adapter, prompts/structured output, moderation, rate/cost limits, and audit.
- Integrate Flutter UX, user feedback/reporting, and transparency copy.
- Run offline evals/red-team tests before small cohort rollout and monitor regressions.

Client must provide or complete

- Create client-owned OpenAI organization/project, add billing/usage limits, invite least-privilege developers, and provide project API key via secret manager.
- Approve exact AI use cases, model/provider, budget, retention/data controls, regions, DPA/legal terms, age/safety policy, and human escalation.
- Provide product-approved prompt tone, prohibited outputs, disclaimers, quality examples, evaluation dataset stripped of sensitive data, and fallback behavior.
- Nominate AI product, safety, privacy/legal, finance, and incident owners.

Implementation sequence

138. Use-case/legal/safety approval and provider project.
139. Build redacted prompt/eval dataset and baseline.
140. Implement backend adapter/audit/rate/cost controls.
141. Integrate UI and run red-team/evals.
142. Limited Remote Config rollout with rollback/fallback.

Testing and acceptance

- Golden-set quality/structure, hallucination, bias/fairness, prompt injection, abuse, sexual/minor safety, PII leakage, timeout/rate limit/provider outage.
- Verify store/retention setting and that prohibited fields never enter requests/logs.
- Cost/latency/concurrency limits and circuit breaker.
- User report/feedback and deterministic fallback end-to-end.

Security and privacy controls

- API key never ships in Flutter; use project-scoped key and spend limits.
- Data minimization/redaction before the provider; no KYC, biometrics, contact info, private messages, or precise location.
- Treat outputs as untrusted: validate, moderate, label, and avoid autonomous entitlement/safety decisions.

Official implementation reference: OpenAI API quickstart

28. Transactional email provider (SES / SendGrid / approved)

Account recovery, verification, receipts, membership lifecycle, safety alerts, support, and operational email delivery.

Priority	P0
Disposition	Choose provider and complete existing SMTP abstraction
Current AMORAA state	Nodemailer/SMTP adapter exists; production refuses to start without SMTP, but credentials are empty and only password-reset email is implemented.
Client dependency	Very High - domain/DNS, sender verification, provider onboarding/billing, legal templates; 2 days to several weeks.
Engineering size	M/L
Primary layer	Backend + MySQL delivery ledger

> **Recommendation:** Keep the provider-neutral mail service but choose SES or SendGrid. Use domain authentication (SPF/DKIM/DMARC), provider webhooks, templated versioning, suppression handling, and separate transactional/marketing streams.

Implementation and connection to AMORAA

SDK/API: AWS SES v2 SDK or SMTP; SendGrid Mail API/SMTP; Nodemailer transport retained behind adapter.

Flutter/web changes: No SDK. Ensure email preferences, resend UX, support links, and user-visible delivery failure states are truthful.

Node.js/Express changes: Provider adapter, template renderer/versioning, queue/retry, idempotency, bounce/complaint/suppression webhooks, unsubscribe/preferences by category, and reconciliation.

MySQL requirements: EmailMessages/NotificationDeliveries (template, recipientHash/userId, providerMessageId, status, attempts, lastError, sent/delivered/bounced/complained timestamps) plus EmailSuppressions; avoid storing rendered sensitive bodies.

Configuration/environment: EMAIL_PROVIDER plus SES region/from/configuration set or SENDGRID_API_KEY; EMAIL_FROM/REPLY_TO; webhook signing secret; SMTP fields only for selected transport.

Webhooks/callbacks: Delivery, bounce, complaint, deferral, open/click only if approved; verify SNS/signature or Event Webhook signature and deduplicate provider event ID.

Development team will implement

- Select provider and implement adapter/queue/templates/delivery ledger.
- Configure authenticated sending domain and production access/reputation controls.
- Implement webhooks, suppression, retry/idempotency, preference categories, and monitoring.

- Create transactional template catalog and accessibility/client rendering QA.

Client must provide or complete

- Choose/create company provider account, billing and owner/admin access; for SES provide AWS account/region and approve production-access request.
- Provide DNS access to add SPF/DKIM/DMARC/custom return-path records and approve From/Reply-To/support addresses.
- Provide business website, legal address, privacy/terms, unsubscribe/contact requirements, brand assets/copy/locales, and transactional vs marketing classification.
- Nominate deliverability, support, privacy/legal, and finance owners; provide test inboxes across Gmail/Outlook/Apple.

Implementation sequence

143. Provider selection/domain authentication.
144. Implement adapter/templates/queue/ledger.
145. Configure webhooks/suppression and non-production stream.
146. Run deliverability/render/security tests.
147. Request/approve production sending and gradual warm-up.

Testing and acceptance

- Send/deliver/bounce/complaint/defer/suppress/retry/duplicate and provider outage.
- SPF/DKIM/DMARC pass; links/from/reply-to work; templates render accessibly on major clients.
- Forgot-password enumeration protection and token expiry remain intact.
- No secrets/OTP/password reset tokens appear in logs; webhook replay rejected.

Security and privacy controls

- API/SMTP credentials in secret manager, rotated and least privilege.
- Suppress bounced/complaining recipients and separate marketing consent.
- Minimize stored body/recipient data; signed links and short-lived recovery tokens.

Official implementation reference: SendGrid sender identity

29. Transactional SMS / WhatsApp provider

Non-OTP transactional alerts, compliant fallback communications, reminders, and support notifications.

Priority	P1
Disposition	Conditional; separate from Twilio Verify OTP
Current AMORAA state	Generic sendSms sends OTP through Twilio Programmable Messaging only; no messaging service, templates, WhatsApp sender, consent ledger, delivery callbacks, or category routing.
Client dependency	Critical for India/WhatsApp - business verification, number/sender/DLT/WABA templates and legal consent; 2-8+ weeks.
Engineering size	L
Primary layer	Backend + MySQL delivery ledger

> **Recommendation:** Use Twilio Messaging/WhatsApp or another approved India provider only for specifically approved transactional categories. Keep OTP in Verify. Do not use SMS/WhatsApp for sensitive dating content or marketing without explicit consent and compliant templates.

Implementation and connection to AMORAA

SDK/API: Provider Messaging API/Node SDK; WhatsApp Business Platform via BSP; India DLT/template/sender integrations as applicable.

Flutter/web changes: Add channel preferences/consent and clear opt-out/help text; do not expose provider keys. Deep links must re-authenticate before showing private content.

Node.js/Express changes: Channel router, template IDs/variables, opt-in/out/quiet hours, messaging service/sender selection, queue/retry, delivery callbacks, status ledger, cost/frequency caps, and fallback rules.

MySQL requirements: MessagingConsents, MessageDeliveries, ApprovedTemplates, ChannelSuppressions; store body/template variables only as minimally required and avoid sensitive content.

Configuration/environment: MESSAGING_PROVIDER, API credentials, messaging service/sender/WhatsApp number, template/content SIDs, callback secret; DLT entity/header/template IDs where required.

Webhooks/callbacks: Delivery status and inbound opt-out/reply callbacks; verify provider signature, deduplicate, and honor STOP immediately.

Development team will implement

- Separate Verify OTP from general messaging and define approved category/channel matrix.

- Implement template-based provider adapter, consent/preferences, queue/status callbacks, rate/cost caps, and fallback.
- Configure WhatsApp/DLT/sender/template approvals and operational dashboards.
- Add opt-out, deep-link privacy, incident, and content QA.

Client must provide or complete

- Select provider/BSP and complete company/business verification, billing, approved phone number ownership, WABA/Meta Business verification for WhatsApp, and India DLT Principal Entity/sender/header/template registrations where applicable.
- Provide legal entity/tax/address/authorized signatory/website/privacy details and secure account invitations/credentials.
- Approve channels, use cases, templates/locales, opt-in evidence, opt-out/help copy, quiet hours, frequency caps, fallback, and monthly budget.
- Nominate messaging compliance, marketing/operations, support, and finance owners.

Implementation sequence

148. Legal/provider/channel approval and registrations.
149. Approve templates/consent/fallback matrix.
150. Implement adapter/ledger/callbacks/preferences.
151. Sandbox and real-number/template testing.
152. Limited production rollout and delivery/cost monitoring.

Testing and acceptance

- Approved/unapproved template, opt-in/out/STOP, quiet hours, invalid number, carrier/provider failure, duplicate/retry, fallback, and account switch.
- Signed callback and replay rejection.
- WhatsApp template window/status and SMS DLT mapping where applicable.
- Lock-screen/message content reveals no sensitive relationship activity.

Security and privacy controls

- Provider secrets backend-only and callbacks signature-verified.
- Explicit consent/opt-out and minimum necessary message content.
- No KYC, exact location, private message text, sexual/religious preferences, or sensitive match details.

Official implementation reference: Twilio WhatsApp Business Platform

30. Customer support / helpdesk provider

Tickets, help center, escalation, identity-aware support, safety triage, and operational reporting.

Priority	P1
Disposition	Choose provider and support operating model
Current AMORAA state	Flutter has local FAQ/support UI; no ticket API, external help center, support accounts, SLA workflow, or SupportTickets table.
Client dependency	Very High - tool procurement, agents/workflows/SLAs/legal access; 1-4 weeks.
Engineering size	L
Primary layer	Flutter + backend/admin + helpdesk

> **Recommendation:** Select Zendesk, Freshdesk, Intercom, or approved equivalent based on support/safety workflow. Use backend-mediated OAuth/service integration; if Zendesk is chosen, design for OAuth rather than long-lived API tokens because token authentication is being retired.

Implementation and connection to AMORAA

SDK/API: Selected helpdesk Tickets/Users/Help Center API, OAuth 2.0/service credentials, webhooks, and optional Flutter/web messenger only if privacy-approved.

Flutter/web changes: Add authenticated contact form, category/severity, attachment consent, ticket reference/status, help-center deep links, and emergency/safety routing; do not expose provider credentials.

Node.js/Express changes: Create/update requester/ticket, attach sanitized files, map user/ticket IDs, verify provider webhooks, enforce rate limits, support deletion/anonymization, and route critical safety cases.

MySQL requirements: SupportTickets (userId, provider, externalTicketId unique, category, priority, status, created/updated/resolved timestamps), SupportTicketEvents, attachment MediaAssets; store minimal duplicate content.

Configuration/environment: SUPPORT_PROVIDER, subdomain/base URL, OAuth client/secret/refresh token or service credentials, webhook signing secret, help center URL.

Webhooks/callbacks: Ticket/status/comment/user events; verify signature/timestamp, deduplicate, and avoid feedback loops.

Development team will implement

- Run provider/workflow decision and map support/safety categories, SLAs, escalation, roles, and data access.
- Implement backend proxy, ticket mappings, sanitized attachments, signed webhooks, and status sync.
- Integrate Flutter contact/status/help-center UX and operational alerts.
- Add retention/deletion/export, audit, rate-limit, and incident runbooks.

Client must provide or complete

- Select/purchase client-owned helpdesk plan and provide organization owner/admin invitations, billing, support domain/subdomain, and brand assets.
- Provide support agent emails/roles, business hours/time zones/languages, categories/forms/macros, SLAs, escalation matrix, safety/emergency procedure, and approval boundaries.
- Provide help-center content, Privacy/Terms/Community Guidelines/Refund/Cancellation links, support/grievance/DPO contacts, and attachment/retention policy.
- Approve OAuth app/service account, webhook configuration, SSO/customer identity, reporting, and who may access sensitive tickets.

Implementation sequence

153. Select provider and design operating model.
154. Configure sandbox/forms/roles/help center/OAuth.
155. Implement backend ticket/webhook mapping and Flutter UX.
156. Run agent, privacy, abuse, attachment, and escalation tests.
157. Train agents and launch with SLA monitoring.

Testing and acceptance

- Anonymous/authenticated create, duplicate/rate limit, attachment type/size/scan, status/comment sync, agent close/reopen, provider outage.
- Signed/replayed webhook, wrong user/ticket access, deletion/anonymization/export.
- Critical safety category reaches correct on-call path; non-emergency copy is clear.
- Support users cannot see unrelated account/KYC/chat data.

Security and privacy controls

- Backend-mediated OAuth/least privilege; no provider secret in app.
- Strict role-based ticket access, audit, redaction, and minimal replicated content.
- Separate emergency/safety escalation from ordinary support and document lawful disclosure process.

Official implementation reference: Zendesk webhook security

Appendix A - Environment variable register

Exact names may be normalized during implementation. Values classified as secrets must be stored in a managed secret store and injected at runtime; they must not be committed, embedded in Flutter, or included in this document.

Firebase/Google	FIREBASE_PROJECT_ID; workload identity/service account; Firebase app config; GA4_MEASUREMENT_ID (public); GA4_API_SECRET; GTM_CONTAINER_ID; SGTM_BASE_URL; Google Ads OAuth/action IDs; Maps restricted keys.
Auth/messaging	GOOGLE_*_CLIENT_ID; APPLE_TEAM/KEY/SERVICE/CLIENT IDs and private key; TWILIO API key/secret + VERIFY_SERVICE_SID; email provider keys; FCM/APNs config; OneSignal IDs if selected.
Payments	Play package/service identity/PubSub; Apple Store API issuer/key/private key; Razorpay key ID/secret/webhook secret.
Acquisition	MMP dev key/app token/S2S token; Branch keys/domain; Meta pixel/dataset/access token; TikTok pixel/access token.
Product providers	Maps server key; Cloudinary cloud/key/secret; AWS region/buckets/distribution/KMS; chat provider key/secret; KYC client/key/cert/webhook; support OAuth/webhook.
Observability/AI	SENTRY_DSN/AUTH_TOKEN; OPENAI_API_KEY/PROJECT/model; release/environment/sampling/budget values.

Appendix B - Recommended new/extended MySQL structures

IntegrationOutbox / DeliveryAttempts	Durable authoritative events, destinations, consent snapshot, retries, provider response, dead-letter.
ProviderWebhookEvents	Provider/event ID unique, payload hash, signature result, environment, received/processed status and timestamps.
ExternalIdentities	Google/Apple/provider subject unique, user mapping, linked/revoked timestamps, relay flags.
AttributionInstalls/Touches	MMP and web campaign first/last touch, approved click IDs, consent and expiry.
StoreEvents + subscription extensions	Play/Apple product/purchase/original transaction identifiers, lifecycle/acknowledgement, unique notification/message IDs.
MediaAssets/Upload Sessions	Provider asset ID/key, purpose/access class, checksum, sizes, moderation, version/deletion.

VenueLocations	Google place ID, name/address/coordinates/categories/source/refresh time; references from events/date spots.
IdentityVerification extensions	Provider reference, method, consent, masked/tokenized result, liveness/decision, expiry; no raw Aadhaar identifier.
AiRequests/AiFeedback	Use case, prompt/model version, hashes, safety/cost/latency/provider request, feedback; minimized content.
SupportTickets/Events	User/provider ticket mapping, category/priority/status/SLA timestamps and minimal sync state.
MessagingConsents/Deliveries/Suppressions	Channel/category consent, template, provider message state, opt-out/bounce/complaint.

Appendix C - Client handover and acceptance checklist

- All production accounts are owned by the client organization; at least two client admins have MFA and recovery access.
- Development access is by named invitation and least privilege; shared passwords and emailed private keys are prohibited.
- A credential register records owner, purpose, environment, location, creation/rotation/expiry, and emergency revocation procedure.
- Final package/bundle/store IDs, signing fingerprints, domains, redirect URIs, and webhook URLs are documented and tested.
- Legal/privacy documents and consent versions cover every enabled SDK/destination; data retention/deletion and DPO/grievance contacts are approved.
- Provider sandboxes and production dashboards have test evidence, alert owners, budgets, quotas, and support contacts.
- Reconciliation, incident, outage, credential rotation, webhook backlog, refund, KYC escalation, support, and rollback runbooks are accepted.
- No unresolved choose-one conflict remains; duplicate SDKs/destinations are disabled before launch.
- Production smoke tests and final go-live sign-off are recorded by engineering plus the relevant client business/legal/finance owner.

Appendix D - Source and review basis

Repository evidence reviewed includes Flutter pubspec/main/config/auth and feature repositories; Android/iOS/web release configuration; Express server/config/routes/controllers/providers/storage/realtime; Sequelize models/migrations; environment example/configuration presence; and the internal production-readiness audit dated 12 August 2026. The report was prepared against commit 2cb0ed59 on 14 August 2026.

Official vendor documentation links are included in each integration section. Key regulatory references include UIDAI requesting-entity security requirements and MeitY's Digital Personal Data Protection Rules, 2025. This engineering report is not a substitute for legal, tax, payment-policy, or Aadhaar/KYC counsel.

MeitY DPDP Rules 2025: Official document page

UIDAI regulations index: Official updated regulations

Appendix E - Final decisions required from the client

158. Final Android/iOS application identifiers, release signing ownership, store organizations, and production/staging domains.
159. Consent/CMP, analytics/ads/MMP data-sharing, retention, ATT, audiences, and prohibited-data policy.
160. AppsFlyer or Adjust; whether its deep links are sufficient or Branch is additionally justified.
161. Direct FCM or OneSignal; no dual production send path.
162. Retain current Socket.IO/MySQL chat or replace it with Stream/Sendbird; if replacing, choose vendor/region/migration policy.
163. Cloudinary for public media, AWS for full stack, or a documented hybrid with KYC/private-media segregation.
164. Approved KYC provider/method and written legal/data-retention decision before raw Aadhaar production processing.
165. Play/App Store digital subscription catalog and Razorpay-eligible web/physical product boundary.
166. Email provider, transactional SMS/WhatsApp scope, India DLT/WABA path, and support/helpdesk provider.
167. OpenAI/LLM use case, model, safety/privacy/budget/retention and launch cohort; or explicit deferral.
168. Server-side GTM hosting/budget and TikTok activation based on funded campaign plans.

> **Ready-to-start definition:** An integration is ready only when the client-owned account is verified and funded if needed, final IDs/domains are available, credentials are transferred securely, legal/consent decisions are approved, test users/devices exist, and a named client approver accepts the provider-specific checklist.